Theory of the dashboard

`wd-magnetizada`

What each tab computes: equations, code
and the decisions that cost real time

July 29, 2026

Magnetized white dwarf with a mixed interior field (poloidal + toroidal)
Technical reference document — not an introduction to stellar physics or MHD

Contents

This document explains what the dashboard computes: the equations behind each number and each figure, and the code that implements them. It is not an introduction to stellar physics or MHD — it assumes the reader already knows that. What this document does is bridge the theory (section 1) and the real code (`scf/`, `dashboard/`).

See `plano_wd_magnetizada.md` for the full project plan (decisions D1–D6, architecture, phases). This document is about the *implemented physics*, not the work plan.

Table of symbols and units

Symbol	Meaning	Unit (CGS)	Name in code
$r,\ \theta$	spherical coordinates	cm, rad	`r`, `theta`
ϖ	cylindrical radius, $\varpi = r\sin\theta$	cm	`omega` (see note)
z	height, $z = r\cos\theta$	cm	`z` (only in `plots.py`)
ρ	mass density	$\mathrm{g\,cm^{-3}}$	`rho`
x	normalized Fermi momentum, $x = p_F/(m_e c)$	dimensionless	`x`
P	pressure	$\mathrm{dyn\,cm^{-2}}$	`pressure(x)`
H	specific enthalpy	$\mathrm{erg\,g^{-1}}$	`H`
Φ	gravitational potential	$\mathrm{erg\,g^{-1}}$	`Phi`
A_φ	φ component of the vector potential	G cm	`A_phi`
u	flux function, $u = \varpi A_\varphi$	$\mathrm{G\,cm^3}$	`u`
B_r, B_θ, B_φ	magnetic field components	G (gauss)	`Br`, `Bth`/`Btheta`, `Bphi`
$f(u)$	poloidal current function, $f(u) = k_0$	$\mathrm{g^{-1/2}cm^{1/2}s^{-1}}$	`k0`
$M(u)$	poloidal Bernoulli potential	$\mathrm{erg\,g^{-1}}$	`M_u` (local in `scf.py`)
$\beta(u)$	toroidal current function, $\beta = \varpi B_\varphi$	G cm	not computed separately
ζ	amplitude of the imposed toroidal field	—	`zeta`
m	exponent of the imposed toroidal field	integer ≥ 1	`m_tor`
u_c	value of u on the last closed line	$\mathrm{G\,cm^3}$	`u_c`
C	Bernoulli constant	$\mathrm{erg\,g^{-1}}$	`C`
ρ_c	central density (input parameter)	$\mathrm{g\,cm^{-3}}$	`rho_c`
μ_e	mean molecular weight per electron, $Y_e = 1/\mu_e$	dimensionless	`mu_e`
M	total stellar mass	g (displayed in $M_\odot$)	`M` (collides with $M(u)$ — see note)
R_{eq}, R_{pol}	equatorial and polar radii (where $H = 0$)	cm (displayed in km)	`R_eq`, `R_pol`
W	gravitational energy	erg	`W`
Π	internal energy, $\int P\,dV$	erg	`Pi`
E_{pol}, E_{tor}, $\mathcal{M}$	magnetic energies	erg	`E_pol`, `E_tor`, `E_mag`
VE	virial error	dimensionless	`VE`
G	gravitational constant	6.674×10^{-8} cgs	`G_CONST`
A, B	EOS constants	see §1.1	`A_CONST`, `B_of_mu_e()`
l	degree of the Legendre expansion	integer ≥ 0/≥ 1	`l`
t_{dyn}, v_A, t_{Alf}	derived scales	s, cm/s, s	`t_dyn`, `v_A`, `t_alf`

Notation notes:

- The code uses the variable `omega` for ϖ (cylindrical radius), **not** for angular velocity — there is no rotation in this project (D3, static star). It is a naming choice specific to this code; whenever you read `omega` in `scf.py`/`gradshafranov.py`, it always means $\varpi = r\sin\theta$.
- M is used in the source theory for two different objects: the total stellar mass and the Bernoulli potential $M(u)$ (§1.5). The code resolves the collision by naming the mass `M` (returned by `scf.total_mass()`) and the potential `M_u` (a local variable inside `hachisu_scf()`, never exposed outside the function). In this document, whenever there is risk of ambiguity, the text spells out "mass" or "Bernoulli potential" in full.
- `Bth` appears as `Btheta` in `diagnostics.py` (function parameter) and `Bth` on the dashboard pages (local variable) — same object, two names.

1 Common theoretical core

1.1 Equation of state

Fully degenerate electron gas at $T = 0$:

$$P = A\left[x(2x^2-3)(x^2+1)^{1/2} + 3\sinh^{-1}x\right] \tag{1}$$

$$\rho = Bx^3 \tag{2}$$

$$x \equiv \frac{p_F}{m_e c} \tag{3}$$

$$A = 6.01\times10^{22}\ \mathrm{dyn\,cm^{-2}} \tag{4}$$

$$B = \frac{9.82\times10^5}{Y_e}\ \mathrm{g\,cm^{-3}} \qquad (Y_e = 0.5 \Rightarrow B \approx 1.96\times10^6) \tag{5}$$

A is a fixed physical constant (a combination of m_e, c, $\hbar$); B depends on composition only through Y_e (electrons per baryon). The dashboard parametrizes by $\mu_e = 1/Y_e$ instead of Y_e directly.

→ `scf/eos.py :: pressure()`

(P(x)), `density()` ($\rho(x)$), `B_of_mu_e()` ($B(\mu_e)$)

Specific enthalpy, analytic — this is what makes the SCF possible:

$$H = \int \frac{dP}{\rho} = \frac{8A}{B}\left[\sqrt{1+x^2} - 1\right] \tag{6}$$

→ `scf/eos.py :: enthalpy()`

Inverse, applied at every SCF iteration:

$$x = \sqrt{\left(1 + \frac{HB}{8A}\right)^2 - 1} \qquad (H \le 0 \Rightarrow \rho = 0) \tag{7}$$

→ `scf/eos.py :: x_of_enthalpy(), density_of_enthalpy()`

The -1 term normalizes $H = 0$ at the surface (the $\rho = 0$ boundary of the star).

Effective polytropic index. Near the surface $x \ll 1$ gives $\rho \propto H^{3/2}$, i.e. $n = 3/2$. In the relativistic core $x \gg 1$ gives $\rho \propto H^3$, i.e. $n = 3$. The EOS slides between the two, and $n = 3$ is the marginal case where the mass becomes independent of the central density — the origin of the Chandrasekhar limit. This fact is the direct cause of the parametrization decision in §1.4/§1.5.

Sound speed. Added after the original theoretical spine (it was not in the plan), because it is needed for the amplitude of the perturbation exported in Tab 3:

$$c_s = \sqrt{\frac{dP}{d\rho}}\,, \qquad \frac{dP}{dx} = \frac{8Ax^4}{\sqrt{1+x^2}} \quad \text{(analytic derivative of } P(x)\text{)} \tag{8}$$

→ `scf/eos.py :: sound_speed()`

, validated against a finite difference in `scf/tests/test_sound_speed.py`

1.2 Self-gravity

$$\nabla^2 \Phi = 4\pi G \rho \tag{9}$$

Solved by expansion in Legendre polynomials. With $\rho(r,\theta) = \sum_l \rho_l(r) P_l(\cos\theta)$:

$$\Phi_l(r) = -\frac{4\pi G}{2l+1}\left[r^{-(l+1)} \int_0^r \rho_l(r')\, r'^{\,l+2}\, dr' \;+\; r^l \int_r^\infty \rho_l(r')\, r'^{\,1-l}\, dr' \right] \tag{10}$$

The weights $r'^{\,l+2}$ and $r'^{\,1-l}$ carry the r'^2 of the volume element.

→ `scf/poisson.py :: solve_poisson()`

(exact implementation of this formula, D_l/E_l computed by cumulative trapezoidal sums); `legendre_matrix()` computes $P_l(\cos\theta)$.

Validated against the closed-form solution of the uniform sphere and Newton's shell theorem in `scf/tests/test_poisson.py`.

$G = 6.674 \times 10^{-8}\ \mathrm{cm^3\,g^{-1}\,s^{-2}}$ — a fixed physical constant (rounded CODATA reference value), not a project parameter.

→ `scf/poisson.py :: G_CONST`

, repeated in `dashboard/units.py :: G_CONST` (same value, two modules, because `poisson.py` cannot depend on the dashboard — R1).

1.3 Magnetic field and the flux function

With $\varpi = r\sin\theta$, the flux function is defined as

$$u = \varpi A_\varphi \tag{11}$$

and the axisymmetric field splits into poloidal plus toroidal:

$$\mathbf{B} = \frac{1}{\varpi}\left[\nabla u \times \hat{e}_\varphi \;+\; \beta(u)\,\hat{e}_\varphi\right] \tag{12}$$

In spherical components:

$$B_r = \frac{1}{r^2 \sin\theta} \frac{\partial u}{\partial \theta} \tag{13}$$

$$B_\theta = -\frac{1}{r \sin\theta} \frac{\partial u}{\partial r} \tag{14}$$

$$B_\varphi = \frac{\beta(u)}{\varpi} \tag{15}$$

→ `scf/diagnostics.py :: poloidal_field()`

implements B_r, B_θ (via finite differences of u, protected against the coordinate singularity at $\varpi \to 0$). B_φ is not computed from an intermediate $\beta(u)$; it comes directly out of `scf/toroidal.py :: impose_toroidal()`, which already implements the chosen functional form for β (§1.9) substituted into the formula above.

The general form $\mathbf{B} = \frac{1}{\varpi}[\nabla u \times \hat{e}_\varphi + \beta \hat{e}_\varphi]$ does not appear anywhere in the code as a vector identity — the code goes straight to the three scalar components.

Geometric interpretation: the contours of u are the poloidal field lines. That's why Tab 1 plots them (§2).

1.4 Grad-Shafranov

$$\Delta^* u = -4\pi\varpi^2 \rho f(u) - \beta\beta'(u) \tag{16}$$

$$\Delta^* = \frac{\partial^2}{\partial \varpi^2} - \frac{1}{\varpi}\frac{\partial}{\partial \varpi} + \frac{\partial^2}{\partial z^2} \tag{17}$$

$f(u) = dM/du$ is the current function generating the poloidal field; $\beta(u)$ generates the toroidal field. The simplest choice, and the one used here, is $f(u) = k_0$ constant (Lander & Jones 2009). Since the toroidal field is imposed after the poloidal field has converged (§1.9, D6) and does not enter the GS equation actually solved by the SCF, the term $-\beta\beta'(u)$ **does not appear in the source the code actually assembles** — the implemented source is only $-4\pi\varpi^2\rho k_0$.

→ `scf/gradshafranov.py :: solve_gradshafranov()`

— source passed by `scf/scf.py :: hachisu_scf()` as `source = -4*np.pi*omega2*rho*k0`.

The current density corresponding to this source is

$$J_\varphi = c\rho\varpi f(u) \tag{18}$$

This formula **does not correspond to any function in the code** — J_φ is never computed as a named quantity. It was used only analytically, during the debugging that found the bug described in the box below, to assemble the right-hand side of the integral Ampère's-law test (derivative-free tests, further below).

Solution by associated-Legendre expansion. The operator Δ^* separates into $P_l^1(\cos\theta)$ functions, **not** $P_l(\cos\theta)$. The radial structure of the Green's function is analogous to that of Poisson, but the integral weights **are not the same**:

	inner weight	outer weight
Poisson (for Φ)	$r'^{\,l+2}$	$r'^{\,1-l}$
Grad-Shafranov (for u)	$r'^{\,l+1}$	$r'^{\,-l}$

→ `scf/gradshafranov.py :: solve_gradshafranov()`

In the lines where `D_l`/`E_l` are assembled with `r ** (l + 1)` and `r ** (-l)`.

Why it's like this — the Green's-function exponents

The difference of one power comes from the ϖ weight of the GS source compared to the r'^2 of the volume element in Poisson, and depends on whether the equation is solved for u or for A_φ. **This was a real bug in this project:** an earlier version of `gradshafranov.py` used $r'^{\,l+2}$ and $r'^{\,1-l}$ (mistakenly copied from the structure of `poisson.py`, which has one extra power of r because of the substitution $\chi = r\Phi_l$ used in reducing the scalar Laplacian — Δ^* does not need that substitution). The bug inflated the poloidal field by a factor of $\sim 7000\times$ and the magnetic energy by $\sim 5 \times 10^7$.
The bug survived piece-by-piece validation — including a "by-hand" closed form — because that closed form was built by reusing the same indicial equation (r^{l+1}, r^{-l} for the homogeneous solutions, which **were correct**) and therefore inherited the same normalization of the internal integrals, which was wrong. Only two tests that use neither the Green's function nor a second derivative — consistency via Ampère's law — revealed the fixed factor. See the full note (with the reasoning chain) at the top of `scf/gradshafranov.py`.

Operational lesson: "solve with the formula, check against the same formula" is not independent validation, even across different files/functions, if both inherit the same normalization from a shared derivation.

Derivative-free normalization tests — the defense against this class of error:

$$\text{Flux:} \quad u(\varpi, z) = \int_0^{\varpi} B_z(\varpi', z)\, \varpi'\, d\varpi' \tag{19}$$

$$\text{Ampère:} \quad \oint_C \mathbf{B} \cdot d\mathbf{l} = 4\pi k_0 \int_S \rho \varpi \, dA \tag{20}$$

- **Ampère**: implemented and in the repository

  → `scf/tests/test_gradshafranov.py :: test_ampere_law()`

  Uses a synthetic case with a pure $l = 1$-mode source (not the full EOS/SCF), computes $\oint \mathbf{B} \cdot d\mathbf{l}$ on a rectangular loop in (r, θ) and compares it against $-\iint [\text{source}/\sin\theta]\, dr\, d\theta$ (an equivalent form derived from Ampère's law for this geometry). Closes to $\sim 2\%$.

- **Flux**: *proposed* during debugging (see the conversation history that fixed the bug) but **not implemented as a separate test** — the Ampère test alone already revealed and confirmed the bug fix, so the flux consistency test was never actually written. Recorded as a gap in §8.

1.5 Bernoulli and the parametrization

$$H + \Phi - M(u) = C\ , \qquad M(u) = \int f(u)\, du \tag{21}$$

With $f = k_0$ constant, $M(u) = k_0 u$.

The integration constant is fixed **at the center**:

$$C = H(\rho_c) + \Phi_c - M(u_c) \tag{22}$$

where Φ_c, u_c here denote the values at the center ($r = 0$) — **note**: this use of u_c (value of u at the center) is different from the u_c used in §1.9 (value of u on the last closed line, at the

surface). They are the same symbol for two different evaluation points; the code never needs both at once, but the reader should note the collision. Since $\varpi = r\sin\theta$ vanishes at the center for any θ, $u_c(\text{center}) = 0$ always and the term $M(u_c)$ disappears — it stays in the formula for generality, not because it contributes.

→ `scf/scf.py :: hachisu_scf()`

— line `C = H_c + Phi[0, 0] - M_u[0, 0]`.

Why it's like this — the (ρ_c, k_0) parametrization

The classical Hachisu recipe fixes C by imposing $H = 0$ at two surface points (pole and equator). This is ill-posed for this EOS: since $n \to 3$ in the core (§1.1), the mass becomes nearly independent of the central density and the radius plunges, so solving for C from the radius inverts a nearly-singular map. Tested and confirmed in this project: the Picard iteration under that recipe is linearly unstable across the whole range $\rho_c = 10^6$–10^{10} g/cm^3, with or without sub-relaxation.

The adopted parametrization is by (ρ_c, k_0), two independent inputs, with no surface condition — C is fixed by a local condition (H at the center), not a global one (an integral over the whole mass). This change alone solved the instability: geometric convergence to machine precision in ∼12 iterations, versus exponential divergence of the old recipe.

Caveat: in the strong-field regime, when the density peak migrates away from the center, this anchor also loses the physical meaning that makes it stable — see §7.

1.6 The SCF loop

1. $\rho \leftarrow$ initial guess (spherical $n = 3$ polytrope)
2. $A_\varphi \leftarrow 0$
3. **repeat:**
   4. $\Phi \leftarrow \text{Poisson}(\rho)$
   5. $A_\varphi \leftarrow \text{GradShafranov}(\rho, f)$
   6. $u \leftarrow \varpi A_\varphi$; $M \leftarrow \int f\, du$
   7. $C \leftarrow H(\rho_c) + \Phi_c - M(u_c)$
   8. $H \leftarrow C - \Phi + M(u)$
   9. $\rho_{new} \leftarrow \text{EOS}^{-1}(H)$ $\qquad [H \le 0 \Rightarrow \rho = 0]$
   10. $\rho \leftarrow (1-\omega)\rho + \omega\rho_{new}$ $\qquad \omega \approx 0.3$
11. **until** $\max|\Delta\rho|/\rho_c < \text{tol}$

→ `scf/scf.py :: hachisu_scf()`

implements steps 1, 3–9 and 11 literally (step 2 is implicit: u starts at zero before the first iteration).

Discrepancy with the real code, step 10: the implementation **does not do sub-relaxation**. The corresponding line in `hachisu_scf()` is `rho = rho_new` — a direct replacement, equivalent to $\omega = 1$, not $\omega \approx 0.3$. There is no ω parameter in the function signature. This is not an oversight: it is a direct consequence of the change described in the §1.5 box. The old recipe (two surface conditions) required sub-relaxation to try to stabilize an iteration that was unstable regardless; the (ρ_c, k_0) parametrization converges by direct replacement because the instability

that sub-relaxation was trying to mask was removed at the root. See the project note at the top of `scf/scf.py` for the full history (including tests showing that sub-relaxing the old recipe did not fix the problem).

The convergence criterion (`tol`) is a numerical, not physical, parameter — the dashboard exposes values from 10^{-4} to 10^{-8} (Tab 1), with 10^{-6} as the default.

1.7 Virial and energy diagnostics

$$W = \tfrac{1}{2}\int \rho\Phi\, dV \qquad \text{(gravitational, negative)} \tag{23}$$

$$\Pi = \int P\, dV \qquad \text{(internal)} \tag{24}$$

$$E_{pol} = \int \frac{B_r^2 + B_\theta^2}{8\pi}\, dV \tag{25}$$

$$E_{tor} = \int \frac{B_\varphi^2}{8\pi}\, dV \tag{26}$$

$$\mathcal{M} = E_{pol} + E_{tor} \tag{27}$$

→ `scf/diagnostics.py :: gravitational_energy()`

(W), `pressure_integral()` (Π), `magnetic_energies()` (E_{pol}, E_{tor}, $\mathcal{M}$ — called `E_mag` in the code). All use `volume_integral()` as their base ($dV = r^2 \sin\theta\, dr\, d\theta\, d\varphi$, φ integrated analytically to 2π).

Scalar virial theorem for a static configuration:

$$W + 3\Pi + \mathcal{M} = 0 \tag{28}$$

Virial error, used as a quality gate:

$$\mathrm{VE} = \frac{|W + 3\Pi + \mathcal{M}|}{|W|} \qquad \text{acceptance criterion: } \mathrm{VE} < 10^{-3} \tag{29}$$

→ `scf/diagnostics.py :: virial_error()`

The 10^{-3} threshold is a project convention (V3 of the plan), not a physical constant — it appears hard-coded as a comparison in `dashboard/pages/1_equilibrium.py` and `3_export.py`, not in `units.py` nor in `diagnostics.py` (no physics module defines this threshold as a named constant — it is a UI/gate decision repeated in two pages).

Magnetic virial identity. Exact, and it is the test that links the magnetic sector to the gravitational one:

$$\int \rho\, \nabla M(u) \cdot \mathbf{r}\, dV \;=\; \int \frac{B^2}{8\pi}\, dV \tag{30}$$

This identity **does not correspond to any function in the code**. It was verified numerically in an ad hoc way during the debugging of the §1.4 bug (comparing $k_0 \int \rho\, r\, \partial u/\partial r\, dV$ with $\int B^2/8\pi\, dV$ for a converged solution), but there is no function in `scf/diagnostics.py` that computes it nor a committed test that exercises it. Recorded in §8 as a gap.

Mandatory conceptual note

$M(u)$ **is not** the magnetic energy. It is the specific potential of the Lorentz force — only the part that enters the Bernoulli equation. There is no *local* identity between $M(u)$ and $B^2/8\pi$. The *global* identity above is what forces the two quantities to agree in order of magnitude. Measured in this project, in the linear regime (perturbative field), the ratio $(E_{mag}/|W|)/(M_u/H_c)$ is ≈ 0.5 and is constant in k_0 (confirmed by doubling k_0: both ratios quadruple, the ratio between them does not change); it drifts to ~ 0.38 as the field stops being a small perturbation ($k_0 \gtrsim 10^{-12}$ in the $\rho_c = 10^9$, $R \approx 3 \times 10^8$ cm configuration). This was the observation that, together with the Ampère test, confirmed that the bug in the §1.4 box was real and not a units artifact.

Physical limit. $\mathcal{M} < |W|$ is a rigid constraint: $E_{mag}/|W| \geq 1$ is an impossible configuration (it would violate the virial with $\Pi \geq 0$). Use it as a sanity anchor when reading any result — if the dashboard shows $E_{mag}/|W|$ above a few tenths, be suspicious before believing it.

1.8 The two Bt/Bp ratios

$$\left(\frac{B_t}{B_p}\right)_{\text{energy}} = \frac{E_{tor}}{E_{pol}} \tag{31}$$

$$\left(\frac{B_t}{B_p}\right)_{\text{amplitude}} = \frac{\max|B_\varphi|}{\max|B_{pol}|} \tag{32}$$

→ `scf/toroidal.py :: bt_bp_ratios()`

They differ by orders of magnitude because the toroidal field is confined to a small volume (§1.9). The literature frequently does not say which one it uses. **The dashboard always shows both, labeled** (Tab 1 and Tab 3).

1.9 The toroidal field and the twisted torus

In the barotropic formulation, $\beta = \beta(u)$: the toroidal function depends only on the flux function. It is this condition that cancels the φ component of the Lorentz force and allows the Bernoulli integral to exist (§1.5).

Geometric consequence: outside the star there is no current, so $B_\varphi = 0$ there; since $\beta = \beta(u)$, it must vanish on every flux line that escapes the surface. **The toroidal field is automatically confined to the region of closed poloidal lines** — the twisted torus is not imposed by geometric decree, it falls out of the consistency of the equation.

Adopted functional form:

$$\beta(u) = \zeta\,(u - u_c)^{m+1}\,\Theta(u - u_c)\ , \qquad m \geq 1 \tag{33}$$

with u_c the value of u on the last closed line (here, u_c = the value of u at the surface — see the notation distinction in §1.5). The exponent ≥ 1 keeps $\beta\beta'$ continuous at the edge of the torus.

→ `scf/toroidal.py :: impose_toroidal()`

implements this form already substituted into $B_\varphi = \beta/\varpi$: $B_\varphi = \zeta(u - u_c)^{m+1}/\varpi$ for $u > u_c$, 0 outside. `find_uc()` implements the search for u_c as the **maximum of u along the entire stellar surface** ($H = 0$, sweeping θ from pole to equator) — it is this specific choice of "last

closed line" that the code uses; contours with u larger than that maximum, by definition, do not touch the surface anywhere and remain entirely interior.

ζ is not a direct user input parameter — the dashboard asks for the target B_t/B_p (energy) ratio and solves for ζ to reach it, exploiting the fact that B_φ is linear in ζ (so E_{tor} is quadratic):

→ `scf/toroidal.py :: solve_zeta_for_energy_ratio()`

Why the toroidal field is imposed rather than solved for

The condition $\beta = \beta(u)$ confines the toroidal field to a small-volume torus, and the resulting energy ratio — if β were extracted from a general barotropic closure condition instead of chosen freely — comes out to a few percent. **It is not possible to reach $B_t/B_p \sim 1/2$ that way.** That's why the project solves the barotropic SCF only for the poloidal field (§1.4–§1.6, with $f(u) = k_0$) and imposes the toroidal field on top, at the desired ratio (ζ solved for the target), loading it into Castro out of exact equilibrium and relaxing it with damping (sponge, Tab 3). For a dynamical study, exact equilibrium is not required — it just needs to be close enough that the transient doesn't destroy the topology.

Torus volume fraction (how much of the star has $u > u_c$):

→ `scf/toroidal.py :: closed_torus_volume_fraction()`

Torus radial extent (used to check mesh resolution, Tab 3):

→ `scf/toroidal.py :: torus_radial_extent()`

, measured along the equator by default.

1.10 Time scales and units

$$t_{dyn} = \sqrt{\frac{R_{eq}^3}{GM}} \tag{34}$$

$$v_A = \frac{\langle B \rangle}{\sqrt{4\pi\bar{\rho}}} \tag{35}$$

$$t_{\text{Alfven}} = \frac{R_{eq}}{v_A} \tag{36}$$

→ `dashboard/units.py :: dynamical_time(), alfven_speed(), alfven_time()`

$\langle B \rangle$ and $\bar{\rho}$ (volume averages) are computed in `dashboard/pages/3_export.py` directly with `diagnostics.volume_integral()` — there is no dedicated `mean_field()` function in `scf/`.

The ratio $t_{\text{Alfven}}/t_{dyn}$ measures the cost of the simulation: in the weak-field regime it reaches 10^3–10^4, which is prohibitive; in the strong-field regime of this project, with $E_{mag}/|W| \sim 0.1$, it stays at 2–3 (D4 of the plan).

Natural field unit. From $B^2/8\pi \sim G\rho^2R^2$:

$$B_{unit} = R\rho\sqrt{8\pi G} \tag{37}$$

For $\rho_c \sim 10^9$ and $R \sim 10^8$ cm this gives $\sim 10^{14}$ G, which is also the order of a white dwarf's virial field. **This formula is not implemented anywhere in the code** — it was used only as an order-of-magnitude estimate during the debugging of the §1.4 bug, to check whether the reported field made physical sense before looking for the numerical error. **Sanity anchor:**

with $E_{mag}/|W| \sim 0.1$, the field shown by the dashboard should be in the 10^{13}–10^{14} G range; if it comes out very different from that, be suspicious before believing it (it was exactly this suspicion that found the bug).

Castro convention: the field is loaded as $B' = B/\sqrt{4\pi}$. The dashboard **always displays gauss** and converts only at export time (Tab 3).

→ `dashboard/units.py :: gauss_to_castro(), castro_to_gauss()`

Checked in this work cycle: the factor matches the plan ($\sqrt{4\pi} \approx 3.5449$). **Status note:** `gauss_to_castro()` exists and is correct, but **it is not called anywhere in the current export pipeline** (`dashboard/pages/3_export.py` writes `B_phi` to the HDF5 file directly in gauss, with an explicit `units` attribute documenting this). Converting to the Castro B' convention is the responsibility of Castro's `problem_initialize.H` (not yet written — Phase 0 of the plan is pending), which will read the HDF5 file in gauss and apply `gauss_to_castro()` (or the C++ equivalent) when assembling the internal state. See §8.

All display-unit conversions (gauss, km) — both the number and the string formatting — live in `dashboard/units.py`, the single source of truth (dashboard rule R4): `cm_to_km()`, `g_to_msun()`, `format_gauss()`, `format_km()`, `format_km_value()`.

1.11 Rotation and the self-consistent toroidal branch

Extension added after the original poloidal-only spine, motivated by the collaboration's interest in $\sim 2\,M_\odot$ white dwarfs as SN Ia progenitors: differential rotation plus a toroidal field, because the two deformations oppose each other (rotation flattens, toroidal elongates), which pushes back the mass-shedding limit.

Architecture. `scf.hachisu_scf()` takes three optional *terms* (`scf/terms/`) instead of one function per physics combination — each term contributes an additive piece to the same Bernoulli equation:

$$H + \Phi - C_{rot}(\varpi) - M_{pol}(u) + M_{tor}(\rho\varpi^2) = C \tag{38}$$

rearranged for the loop: $H = C - \Phi + C_{rot}(\varpi) + M_{pol}(u) - M_{tor}(\rho\varpi^2)$. `rotation`/`poloidal`/`toroidal` = `None` turns a term off (zero contribution, zero energy). `poloidal` and `toroidal` are mutually exclusive (out of scope: self-consistent mixed field — same reason as D6, barotropy does not deliver a free Bt/Bp). With all three `None`, the loop reproduces the pre-extension code **bit for bit** (checked against a frozen copy of the old `scf.py`, `scf/tests/test_regression_v0.py`).

Rotation (`scf/terms/rotation.py`): j-constant law,

$$\Omega(\varpi) = \frac{\Omega_c A^2}{A^2 + \varpi^2} \tag{39}$$

with rigid rotation as the $A \to \infty$ limit, handled analytically (not a large-but-finite A substitute). Its Bernoulli potential has a closed form,

$$C_{rot}(\varpi) = \frac{\Omega_c^2 A^2}{2} \frac{\varpi^2}{A^2 + \varpi^2} \tag{40}$$

reducing to $\frac{1}{2}\Omega_c^2\varpi^2$ as $A \to \infty$.

The virial theorem gains the rotational kinetic term:

$$2T + W + 3\Pi + \mathcal{M} = 0\ , \qquad T = \tfrac{1}{2}\int \rho\,\Omega^2(\varpi)\,\varpi^2\,dV \tag{41}$$

$$\mathrm{VE} = \frac{|2T + W + 3\Pi + \mathcal{M}|}{|W|} \tag{42}$$

→ `diagnostics.virial_error()`

(now takes `T=0.0`, default reduces exactly to the old formula) and `diagnostics.virial_error_terms()` (the term-aware wrapper — pulls $T/B_r/B_\theta/B_\varphi$ from whichever terms are active and calls `virial_error()`, so the residual formula itself lives in exactly one place).

Stability/termination diagnostics:

- $T/|W|$: ≥ 0.14 secular non-axisymmetric instability threshold (Ostriker & Bodenheimer 1973), ≥ 0.27 dynamical bar-mode threshold. Same traffic-light convention as VE; blocks export (Tab 3) at ≥ 0.14, same as VE's R5.
- Equatorial mass-loss ratio: $\Omega^2(R_{eq})R_{eq}/(d\Phi/dr$ at R_{eq}, equator$) \longrightarrow 1$ signals the effective gravity vanishing at the equator (Keplerian breakup).

  → `diagnostics.equatorial_mass_loss_ratio()`

Self-consistent purely toroidal field (`scf/terms/toroidal_sc.py`): $B_\varphi = K\rho^m\varpi^{2m-1}$, $m \geq 1$. Derived from the Lorentz force (Maxwell stress tensor, verified symbolically with SymPy for $m = 1, 2, 3/2$ — residual exactly 0; `scf/tests/test_toroidal_sc.py`):

$$M_{tor}(s) = \frac{mK^2}{4\pi(2m-1)}\, s^{2m-1}\,, \qquad s = \rho\varpi^2 \tag{43}$$

Unlike $M_{pol}(u)$, M_{tor} depends on ρ — the *unknown* being solved for at each grid point — so step 9 of the SCF loop (§1.6) stops being a direct EOS inversion and becomes a per-point root find $(H(\rho) + M_{tor}(\rho\varpi^2) = \text{RHS}$, monotonic increasing in ρ, safe to bracket with `scipy.optimize.brentq` —

→ `scf.py :: _solve_rho_implicit()`

). This branch does **not** use Grad-Shafranov at all — no flux function, no Δ^*, no Green's function; B_φ is an algebraic function of the local ρ, not a PDE solution.

Validated: turning on the field at fixed ρ_c increases the mass (sign check); pure toroidal field produces **prolate** deformation ($R_{pol} > R_{eq}$, opposite of the poloidal/oblate case), VE closes ($\sim 4 \times 10^{-4}$).

Correction — the sign of the magnetic virial identity, and why it is not "M_{tor} depends on ρ"

The magnetic virial identity for this branch is

$$\int \rho\, \nabla M_{tor} \cdot \mathbf{r}\, dV \;=\; -\int \frac{B_\varphi^2}{8\pi}\, dV \tag{44}$$

a **minus** sign — confirmed by an independent derivation (Maxwell stress tensor divergence: $\int \mathbf{r}\cdot\mathbf{f}_L\, dV = +\int B^2/8\pi\, dV$ in general, for any purely toroidal field, regardless of any Bernoulli convention) and by clean numerical convergence with grid refinement ($2.27\% \to 1.00\% \to 0.36\% \to 0.14\%$ at $nr = 65 \to 257$).

The first version of this note attributed the sign to "M_{tor} depends on ρ, M_{pol} does not." **That attribution was wrong**, caught on review. Here is the actual mechanism: writing the master Bernoulli as $H + \Phi - C_{rot} - M_{pol} + M_{tor} = C$ (note the signs: M_{pol} enters with a minus, M_{tor} with a plus) and comparing its gradient against the momentum equation gives $\mathbf{F}_{L,tor}/\rho = -\nabla M_{tor}$ — the minus comes directly from M_{tor} having been written with a **plus** sign in the master equation, full stop. Substituting into the sign-fixed, convention-free Maxwell identity $\int \mathbf{r}\cdot\mathbf{F}_L\, dV = +\int B^2/8\pi\, dV$ is what produces the minus sign in the virial identity. Had the master equation instead been written with $-M_{tor}$, the virial

identity would come out +, with M_{tor} depending on ρ in exactly the same way either time. **ρ entering M_{tor} is why the SCF inversion becomes implicit (a fact about the algorithm's structure) — it has nothing to do with which sign the virial identity carries (a fact about which convention was chosen when the term was first written into the Bernoulli equation).** Two different questions, easy to conflate, and conflating them is exactly the kind of mistake this project has already been burned by once (§1.4's Green's-function bug survived because independent-looking checks secretly shared one derivation). Get the causal story right here or a future reader "fixing" the (correct) code to match the (wrong) causal explanation is a real, specific risk — see `scf/terms/toroidal_sc.py` for the full derivation.

V-R1/V-R2 status (rigid vs. differential rotation, no field). V-R2 (differential, j-constant, $A = 0.3\,R_{guess}$) reaches $2.19\,M_\odot$ against the $\sim 2.2\,M_\odot$ target (Yoon & Langer 2005) — **0.40% error**, reached with mass_loss_ratio ~ 0.13, comfortably away from breakup. `scf/tests/test_differential_rotation.py`.

V-R1 (rigid) **does not validate** against the $\sim 1.5\,M_\odot$ target (Hachisu et al. 2012; Boshkayev et al. 2013). At $\rho_c = 10^{12}$, stepping Ω_c upward with continuation, the sequence terminates *numerically* at $R_{pol}/R_{eq} \approx 0.932$, $\Omega_c \approx 26.47$, with **mass_loss_ratio only 0.135** — far from the $\to 1$ that a genuine mass-shedding termination requires, and far from the Roche-model expectation for a centrally-condensed ($n \approx 3$) configuration, $R_{pol}/R_{eq} = 2/3 \approx 0.667$ at real breakup. The cause is not resolved. Two suspects, not yet investigated (not on the project's critical path — Jorge wants differential rotation, which already works): outer-layer radial resolution, and — most likely — Ω_c itself being a bad control parameter this close to the sequence's end (a documented phenomenon in the rotating-SCF literature; Hachisu's own method sidesteps it by parametrizing on axis ratio and solving for Ω^2, not the reverse). A direct test confirms this: bisecting for a target axis ratio instead of imposing Ω_c directly saturates at the same point — no smaller axis ratio is reachable by continuation in Ω_c, for any target. The mechanism itself is not in doubt: $(M - M_0)/M_0$ tracks $T/|W|$ with a stable proportionality coefficient (~ 3.0) across the whole tested range, and V-R6 (below) independently confirms T. See `scf/tests/test_rotation.py` for the full numbers and reasoning chain.

Bug found and fixed during this investigation (unrelated to the V-R1 question above, but real and previously unnoticed)

`diagnostics.surface_radius()` (and everything built on it — `equatorial_polar_radii()`, `surface_field()`, `surface_dipolarity()`, `toroidal.find_uc()`, `impose_toroidal()`) used to take ρ and linear-interpolate where it crosses zero. But $\rho = \mathrm{EOS}^{-1}(H)$ is clipped to *exactly* 0.0 for $H \le 0$ (`eos.density_of_enthalpy`) — so the grid point just past the surface always has $\rho = 0.0$ exactly, which makes the interpolation fraction collapse to exactly 1.0 and the function always return a raw grid point, never a true sub-grid position. Radii were silently grid-quantized project-wide. Fixed by interpolating on H instead (continuous, crosses zero smoothly between grid points, never clipped) — verified to shift R_{eq}/R_{pol} by tens of km on a modest mesh ($nr = 65$), not a cosmetic correction. Found by noticing that a rotation sequence's R_{pol}/R_{eq} was landing on suspiciously exact small integer ratios (37/39, 38/39, …) — the classic signature of grid quantization, not physics.

T (Fase 1, V-R6). Computed two independent ways — once via `rotation.energy()`, once via a from-scratch volume integral written in the test, not calling any shared code beyond $\Omega(\varpi)$ — and the two agree to machine precision (rel_diff $= 0.00 \times 10^0$).

1.12 Validity gate: ρ_c and inverse beta decay

ρ_c was, until now, the only input with no validity gate — VE and $T/|W|$ already have traffic lights, ρ_c did not, and the slider went up to 10^{12} with no warning even though matter neutronizes long before that density: past the inverse-beta-decay (electron capture) threshold, the constant-μ_e cold EOS used throughout this project (§1.1) stops describing a real white dwarf — composition changes locally, and the resulting configuration (radius of order tens of km at these densities) is not a white dwarf in any recognizable sense (see the Das & Mukhopadhyay 2.58 $M_\odot$/70 km example in `plano_wd_magnetizada.md`).

Threshold densities, from **Table 2 of Boshkayev, Rueda, Ruffini & Siutsou (2013), ApJ 762, 117** (threshold energies from Audi et al. 2003, critical density via the RFMT EOS — verified against the published table, not re-derived here):

Nuclide	$\mu_e = A/Z$	$\rho_{threshold}$ (g/cm^3)
^{4}He	2.0	1.39×10^{11}
^{12}C	2.0	3.97×10^{10}
^{16}O	2.0	1.94×10^{10}
^{56}Fe	$56/26 \approx 2.154$	1.18×10^{9}

μ_e does not uniquely fix composition — ^{4}He, ^{12}C and ^{16}O all have $\mu_e = 2$ exactly but different thresholds, because the capture threshold depends on the specific nuclide's binding energy, not μ_e alone. This project has no composition selector beyond μ_e, so `neutronization_threshold_rho_c(mu_e)` returns the **conservative** (lowest, ^{16}O-like) value for $\mu_e = 2$ — better to warn too early than too late — and linearly interpolates toward the ^{56}Fe point for μ_e up to 56/26; outside [2.0, 2.154] it returns the nearest tabulated value (flat extrapolation, no invented trend).

→ `scf/eos.py :: neutronization_threshold_rho_c()`

Same traffic-light convention as VE/$T/|W|$: warning in Tab 1, blocks export in Tab 3 (R5), no override.

Practical consequence: the dashboard's own default ($\rho_c = 10^{12}$, chosen so the field-free case reproduces the Chandrasekhar limit within 1%, V1) is itself **above** the ^{16}O threshold (1.94×10^{10}) — that default was always an asymptotic numerical convenience (the $M(\rho_c)$ curve for the idealized $T = 0$ free-electron-gas EOS saturates toward $1.44\,M_\odot$ only as $\rho_c \to \infty$, ignoring neutronization), never a claim that $\rho_c = 10^{12}$ is a physically realizable white dwarf. **Physically valid ρ_c for this EOS, standard C/O or He composition, is $\lesssim 2 \times 10^{10}$ g/cm^3** (^{16}O; up to $\sim 1.4 \times 10^{11}$ for pure ^{4}He) — V1's high-ρ_c sequence is a numerical asymptote check, not a claim about real stars, and any run intended for Castro export needs ρ_c inside the physical range.

1.13 Sweep of the ρ-based surface-criterion bug class

The `surface_radius()` bug (§1.11) is structural, not a one-off: *any* logic using $\rho = 0/\rho > 0$ to locate or test the stellar surface is susceptible, because the EOS clip (`density_of_enthalpy`) produces an *exact* zero and destroys sub-cell position information; H crosses zero continuously and does not. Every ρ-based comparison in the codebase was reviewed for this specific failure mode:

Location	Verdict	Why
`surface_radius`, `equatorial_polar_radii`, `surface_field`, `surface_dipolarity`, `toroidal.find_uc/_surface_u_profile/ impose_toroidal/solve_zeta_for_energy_ratio`	**Migrated to H**	Was searching for a sub-grid crossing position — exactly the failure mode
`equatorial_mass_loss_ratio`	Already correct	Takes R_{eq} as an argument (from the now-fixed chain above); interpolates the continuous field $d\Phi/dr$, never touches ρ directly
`toroidal.check_uc_tangency` (vacuum = $\rho \leq 0$)	Adequate, kept	Pointwise boolean membership at *existing* grid points; $\rho \leq 0 \Leftrightarrow H \leq 0$ exactly everywhere by construction, no interpolation involved
`toroidal.torus_radial_extent` ($\rho[:, j] > 0$)	Adequate, kept	Deliberately reports grid-cell radii (Castro cell-counting check) — quantization is the intended semantics
`toroidal.closed_torus_volume_fraction` (inside_star = $\rho > 0$)	Adequate, kept	Feeds a volume integral (grid-summation quadrature); sub-cell precision isn't part of that quadrature's design regardless of boundary criterion
`terms/toroidal_sc.py::B_phi()` (ρ_{safe})	Adequate, kept	Numerical safety clamp before a fractional power, not surface detection
`dashboard/plots.py::plot_flux_contours` (inside_star = $\rho > 0$)	Adequate, kept	Purely a plot line-style choice (solid vs. dashed); the actual boundary drawn in the same figure correctly uses H
`diagnostics.density_peak_location` (argmax(ρ))	Adequate, unrelated	Finds an *interior* maximum (exact discrete argmax, no interpolation) — not a boundary crossing
Domain-overflow detection	**Does not exist**	Not an instance of this bug class (not an interpolation-precision issue — a "should this radius even be trusted" question), but a related gap surfaced by the same investigation (the branch-jump seen chasing V-R1, above) — not implemented here, flagged in §8

Regression test (generic, not tied to one function): `scf/tests/test_surface_radius_continuity.py` scans Ω_c continuously and checks R_{eq} shows no grid-plateau signature. Confirmed discriminating: a standalone reimplementation of the old ρ-based formula gives only 23% unique values across the scan (staircase); the current H-based code gives 100% (continuous).

2 Tab 1 — Equilibrium

→ `dashboard/pages/1_equilibrium.py`

Runs a single SCF pass (§1.6) and shows the result. Uses the entire theoretical core (§1.1–1.10).

Input parameters (sidebar): ρ_c, μ_e, k_0 (optional, toggles the poloidal field on/off), target B_t/B_p ratio and m (toroidal, optional), plus the numerical mesh parameters (N_r, N_θ, l_{max}, tol, max_iter). θ always covers the full $[0, \pi]$ — no equatorial symmetry, a plan decision (D3) so as not to mask asymmetric modes ($m = 1$).

k_0 range. Not known a priori (it depends on ρ_c, R, μ_e in a non-trivial way — see the §1.4 bug, which for a long time made the apparent range look wrong by 3–4 orders of magnitude). The dashboard probes empirically by raising k_0 geometrically until VE $> 10^{-3}$ or the SCF stops converging, on a coarse mesh, and stores the result in a cache in `dashboard/k0_range_cache.json`, indexed by (ρ_c, μ_e).

→ `dashboard/pages/1_equilibrium.py :: _estimate_k0_max()`

Displayed scalars ("Scalars" table):

Scalar on screen	Definition	Function
`M/M_sun`	total mass / `M_SUN`	`scf.total_mass()`, `units.M_SUN`
R_{eq}, R_{pol} (km)	radius where $\rho \to 0$, at the equator and the pole	`diagnostics.equatorial_polar_radii()`
R_{pol}/R_{eq}	flattening	direct ratio
ρ_c confirmed	$\rho[r{=}0]$ post-convergence	should match the input ρ_c
mean ρ	M/volume of the ellipsoid $(R_{eq}, R_{eq}, R_{pol})$	geometric approximation, not an exact integral
W, E_{int}=Π, E_{mag}, E_{pol}, E_{tor}	§1.7	`diagnostics.virial_error()`, `magnetic_energies()`
$E_{mag}/\lvert W\rvert$	dimensionless field strength	direct ratio (sanity anchor, §1.7/1.10)
$B_{pol,max}$, $B_{central}$, $B_{tor,max}$	field values, gauss	maxima/pointwise values of `Br`, `Bth`, `Bphi`
torus volume fraction	§1.9	`toroidal.closed_torus_volume_fraction()`
VE	§1.7	`diagnostics.virial_error()`

Convergence: two plots (max $|\Delta\rho|/\rho_c$ and VE, both vs. iteration, log scale)

→ `plots.plot_convergence(), plots.plot_virial_history()`

The VE history only exists if `hachisu_scf(..., track_virial=True)` — it costs extra integrals every iteration, so it is optional (see §1.6).

Bt/Bp: the two ratios of §1.8, side by side, always labeled.

Meridional-plane figures (§1.3):

- **density**: color map of ρ, with the $H = 0$ boundary (the stellar surface) overlaid in cyan

  → `plots.plot_density()`

- **poloidal field lines**: contours of u — physically, each contour *is* a poloidal field line (§1.3), because $\mathbf{B}$ is tangent to the curves of constant u by construction ($\mathbf{B} \cdot \nabla u = 0$ follows directly from the formulas for B_r, B_θ in terms of derivatives of u). The last closed line (u_c, §1.9) is highlighted in red when a toroidal field is imposed

  → `plots.plot_flux_contours()`

- **toroidal field**: color map of B_φ, shows the confined torus

  → `plots.plot_toroidal()`

All radial axes in km, field always in gauss with a colorbar in scientific notation (rule R4, §1.10).

3 Tab 2 — Sweep

→ `dashboard/pages/2_sweep.py, dashboard/sweep_worker.py`

Runs the SCF (§1.6) on a grid of up to two selectable axes in parallel (`ProcessPoolExecutor`), with caching by parameter hash (`store.run_exists()`/`store.save_run()`, §5). Each grid point is an independent call to `scf.hachisu_scf()` followed by the same diagnostics as Tab 1 — no new physics, just orchestration.

→ `dashboard/sweep_worker.py :: run_one()`

— the picklable function each grid worker process runs.

Selectable axes (extended for rotation/toroidal, `SCHEMA_VERSION=3`). The page carries the same mode selectors as Tab 1 (rotation: none/rigid/differential; field: none/poloidal/toroidal self-consistent, §1.11). The sweep axes on offer follow from those modes: ρ_c is always available; k_0 only under `field=poloidal`; K only under `field=toroidal (self-consistent)`; Ω_c only when rotation is on. For differential rotation, A/R_{eq} is **always fixed** to a single value (never a sweep axis) — opening it as a third axis would multiply the grid size by the axis resolution for no commensurate gain, given the closeout plan's time budget. Every axis not chosen as one of the (at most two) swept dimensions gets a single fixed value from the sidebar instead of being silently zeroed.

Points past the ρ_c validity gate (§1.12) are marked, not filtered. `sweep_worker.run_one()` returns `rho_c_valid` (a boolean, `False` once ρ_c reaches the inverse-beta-decay threshold for the run's μ_e) on every converged row. Every plot on this page that plots against ρ_c distinguishes those points visually (a red × marker on the priority plot, a distinct symbol on the M-R diagram) instead of dropping them — consistent with the rest of the dashboard's rule of showing out-of-validity results rather than hiding them silently.

Priority plot: M vs ρ_c, one curve per K. This is the figure meant to go to a collaborator working with the self-consistent toroidal branch: mass as a function of central density, one line per toroidal-field strength K (including the $K = 0$, field-free reference curve), with a dashed vertical line at the ρ_c inverse-beta-decay threshold for the run's μ_e (§1.12). It renders first on the page, before any other plot, and only needs ρ_c swept with more than one value to be meaningful (it still renders with a single $K = 0$ curve outside `field=toroidal (self-consistent)` mode, degrading gracefully to a plain M vs ρ_c plot).

What an equilibrium sequence is. Fixing ρ_c and varying k_0 (or vice versa) produces a sequence of equilibrium configurations. **The sequence ends** when the SCF stops converging or when VE exceeds the acceptance threshold (§1.7) and does not improve with resolution — see the measured numbers in §7. A sequence ending is, in itself, a physical result (not an error to be fixed): it signals the limit of validity of that family of equilibria.

M-R diagram. R_{eq} (km) on the x-axis, $M/M_\odot$ on the y-axis, colored by k_0. The horizontal line at $1.44\,M_\odot$ marks the Chandrasekhar limit **without a field** ($\mu_e = 2$) — it is not the physical limit of the magnetized sequence, it's just a reading reference. Optional overlay of literature data points from `dashboard/data/references/bera_bhattacharya_2014.csv`, if the file exists (it does not currently exist in this repository — no data has been digitized; the dashboard works without it and warns that it's missing).

Why the relevant mass maximum is over the whole plane, not a slice

This is the point that most often causes an incorrect comparison with the literature. $M_{max}(k_0{=}0)$ already depends on ρ_c: it only approaches the Chandrasekhar limit asymptotically, for high ρ_c ($\sim 10^{11}$–10^{12} g/cm^3 — see `scf/tests/test_scf_v1.py`, validated to 0.78% in that range). A slice at low ρ_c (for example $\rho_c = 10^9$, used during this project's debugging) gives $M(k_0{=}0) = 1.39\,M_\odot$ — **already below** the field-free Chandrasekhar limit, and any mass measured along that slice (with or without a field) is not comparable to the literature reference number ($M_{max} \sim 1.9\,M_\odot$ in Bera & Bhattacharya 2014), which is the maximum taken over the **entire** (ρ_c, k_0) plane, not over a line with k_0 varying at fixed ρ_c. This tab's sweep is exactly what allows that comparison to be made correctly — by mapping the plane, not a slice.

Grid convergence. Points that fail to converge are recorded (not silently dropped) and shown in a separate expander; the grid's convergence rate is, itself, information about where the family of solutions ends.

VE heat map. Over the (ρ_c, k_0) grid, in $\log_{10}$(VE) — reveals where the method is at the edge of validity without having to read numbers one by one.

4 Tab 3 — Export

→ `dashboard/pages/3_export.py`

Takes an already-saved equilibrium (Tab 1 or 2), imposes the toroidal field (§1.9) and generates the three output artifacts: an HDF5 initial data file, Castro's `inputs`, and `run_manifest.json`.

Toroidal imposition. Same function as §1.9 (`toroidal.solve_zeta_for_energy_ratio()`), with this tab's own controls (the target ratio may differ from the one used when the equilibrium was saved). Continuity of $\beta\beta'$ at the torus edge is **guaranteed analytically** for $m \geq 1$ (it is not recomputed numerically here — it is a property of the functional form of §1.9, $(u-u_c)^{m+1}$ and its derivative go to zero at $u = u_c$ for any $m \geq 1$); the page only confirms that `m_tor`≥ 1 was respected.

Why the field is initialized from the vector potential, not from B at cell centers. This is a Castro-side decision (D3 of the plan, section 5), not something the SCF resolves — but the dashboard already prepares the data in that format: the exported HDF5 file contains A_φ (computed as u/ϖ from the converged flux function), **not** B_r, B_θ directly. Castro's constrained-transport algorithm keeps $\mathbf{B}$ on the mesh faces; initializing via $\nabla \times \mathbf{A}$ interpolated onto the edges guarantees $\nabla \cdot \mathbf{B} = 0$ to machine precision by construction. Initializing with $\mathbf{B}$ values at cell centers does not give that guarantee. **What's missing:** interpolating A_φ onto the Cartesian mesh edges and computing $\nabla \times \mathbf{A}$ there is the responsibility of Castro's `problem_initialize_mhd_data.H`, which has not been written yet (Phase 0 of the plan is pending) — the dashboard delivers A_φ on a spherical (r, θ) mesh; interpolation onto Castro's Cartesian mesh happens on the other side.

$B' = B/\sqrt{4\pi}$ convention. See §1.10. The HDF5 file exported by this tab stores B_φ in pure gauss, with a `units` attribute in the header saying so explicitly — the conversion to the Castro convention does not yet happen in this pipeline (see the status note in §1.10 and the gap in §8).

Derived scales and simulation cost. t_{dyn}, v_A, $t_{\rm Alfven}$ and the $t_{\rm Alfven}/t_{dyn}$ ratio from §1.10, computed from the $\langle B \rangle$ and $\bar{\rho}$ of the loaded equilibrium. A success badge appears when the ratio is between 0.3 and 3 — the strong-field regime that makes the simulation cheap (D4 of

the plan).

Torus resolution check. Given the number of cells per side of Castro's box (128/256/384) and the box size (a multiple of R_{eq}), computes how many cells cross the torus's radial extent (`toroidal.torus_radial_extent()`, measured at the equator). Fewer than 10 cells triggers a warning — the torus is small compared to the star and it is the torus that needs to be resolved (plan, section 6).

Box parameters. `castro.small_dens` is chosen as $\rho_c \times 10^{-8}$ — this is a **convention of this dashboard's design**, not a value derived from physics nor cited in the plan; there is no documented theoretical justification for the 10^{-8} exponent beyond it being a density floor low enough not to perturb the star and high enough not to generate an absurd v_A in the numerical vacuum (the "fluff problem", plan section 5). The sponge start radius ($1.5\,R_{eq}$), the damping duration ($5\,t_{dyn}$) and the perturbation amplitude ($10^{-4}c_s$, using `eos.sound_speed()` at the center) come directly from the values cited in the plan (section 5), and are not recalculated/optimized by this dashboard.

R5 — export blocked if VE $\geq 10^{-3}$, with no override option. Implemented as a simple `if/else` around the export button — there is no override mechanism at any layer.

5 Tab 4 — Runs

→ `dashboard/pages/4_runs.py, dashboard/store.py`

No physics — this module only persists what the other tabs have already computed (dashboard rule R1/R3).

What gets saved, per run, in `dashboard/runs/<hash>/`:

File	Content	Function
`params.json`	full input parameters	`store.save_run()`
`scalars.json`	derived scalars (same as the Tab 1 table)	same
`fields.npz`	`rho, Phi, u, H, Bphi, r, theta` on the mesh	same
`manifest.json`	hash, timestamp, git, dependencies	same

`dashboard/runs/index.csv` aggregates hash + parameters + scalars for all runs, so Tab 4 can load quickly without opening every directory (`store.load_index()`).

Why the git hash matters. `manifest.json` records `git_commit_hash(REPO_ROOT)` — the commit of `scf/` **and** `dashboard/` (the same git repository covers both, initialized specifically to give this dashboard provenance; `amrex/`, `castro/`, `microphysics/` are excluded via `.gitignore`, being their own repositories). A scalar without its associated commit cannot be reproduced with confidence — if the code changes (for example, the §1.4 bug being fixed), results saved before and after **are not comparable**, even with the same input parameters. `git_dirty()` is also recorded — it flags whether there were uncommitted changes at the time of the run, which makes exact reproduction impossible even knowing the hash.

→ `dashboard/store.py :: git_commit_hash(), git_dirty(), dependency_versions()`

(versions of `numpy`, `scipy`, `streamlit`, `plotly`, `h5py`, `python`).

Cache. The hash is `sha256(json(params, sort_keys=True))[:12]` (`store.params_hash()`) — deterministic in the parameters, used both to name the run's directory and for Tab 2 to skip already-computed points.

Schema version: fixed — `store.SCHEMA_VERSION` now guards `run_exists()`, so a schema mismatch is a cache miss instead of a silent hit with missing columns. Added after the set of scalars changed during this project more than once (for example, when `B_pol,max (G)` was added to Tab 2's schema, and again when rotation/toroidal_sc added `T`, `T/|W|`, etc.) and runs saved before each change ended up with missing columns in `index.csv`, breaking charts that expected the new column — this actually happened during development and was worked around by manually deleting the old cache before the version guard existed.

Features: filterable/sortable table (`st.data_editor`, formatted per column per rule R4 — gauss in scientific notation, km with 2 decimal places), side-by-side comparison of two runs, reload a run into Tab 1 (via `st.session_state["reload_run_params"]` + `st.switch_page`), mark as reference (`store.mark_reference()` — today it only records the flag; no other tab reads `reference` yet, see §8).

6 Results

The infrastructure built for the post-`fab74b0` closeout (rotation + self-consistent toroidal terms, the ρ_c validity gate, the Tab 2 grid extension) exists to answer one physics question: **is the $\sim 2\,M_\odot$ target reachable within the ρ_c range this EOS can actually claim to describe (§1.12)?** This section reports two direct investigations run against that question, both using the real solver/sweep code paths (not estimates) — the first result to come out of this session that is about the star, not about the tool.

6.1 Does a self-consistent toroidal field extend or shorten the rotation limit?

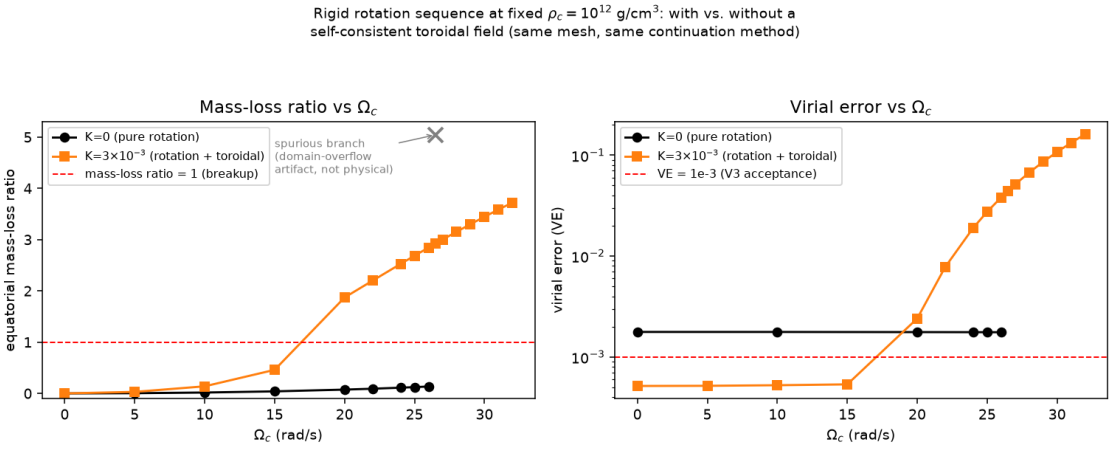
Rigid rotation alone ($K = 0$) is known not to validate against the literature (§1.11, V-R1): the Ω_c-controlled sequence at $\rho_c = 10^{12}$ terminates numerically near $\Omega_c \approx 26$, with mass-loss ratio only 0.135 — far from the $\to 1$ a real mass-shedding termination requires. Before this section, it was an open question whether combining rotation with a self-consistent toroidal field (which favors a **prolate** deformation, opposite to rotation's oblate one, §1.11) would push that numerical limit to higher Ω_c, since the two deformations act in opposite directions on the star's shape.

It does the opposite. Continuation in Ω_c was run twice, same mesh ($n_r = 129$, $n_\theta = 65$, domain $2\times R_{guess}$) and same $\rho_c = 10^{12}$, so the comparison is apples-to-apples: once with $K = 0$, once with a moderate self-consistent toroidal field ($K = 3 \times 10^{-3}$, $m = 1$ — chosen by probing as the smallest K that already produces a substantial static deformation, $E_{tor}/|W| \approx 0.18$ at $\Omega_c = 0$, while still converging cleanly, $VE = 5.2 \times 10^{-4}$).

Ω_c (rad/s)	K=0: mlr	K=0: VE	K=3×10^{-3}: mlr	K=3×10^{-3}: VE
0	0.000	1.78×10^{-3}	0.000	5.18×10^{-4}
10	0.017	1.77×10^{-3}	0.139	5.27×10^{-4}
15	0.040	—	0.460	5.37×10^{-4}
20	0.074	1.77×10^{-3}	**1.871**	2.41×10^{-3}
24	0.112	1.77×10^{-3}	2.525	1.90×10^{-2}
26	0.135	1.76×10^{-3}	2.844	3.82×10^{-2}
26.5	*5.04 (spurious branch)*	—	2.922	4.44×10^{-2}
32	*(not reached)*	—	3.722	1.61×10^{-1}

(mlr = equatorial mass-loss ratio. Full per-point numbers, including $M/M_\odot$ and $q = R_{pol}/R_{eq}$,

are in the investigation scripts this table was built from — kept here to the two reliability diagnostics used throughout this project.)



With $K = 0$, mass-loss ratio grows slowly and stays at 0.135 even at $\Omega_c = 26$ — nowhere near breakup — while VE stays flat at $\approx 1.8 \times 10^{-3}$ regardless of Ω_c (a mesh-resolution offset from this particular domain choice, not a rotation effect; it does not move with Ω_c). The $\Omega_c = 26.5$ point jumps to mass-loss ratio 5.04 — the same spurious branch / domain-overflow artifact already flagged in §1.13's sweep and §8's open questions, not a physical result.

With $K = 3\times10^{-3}$, both diagnostics **fail together and smoothly**, between $\Omega_c = 15$ and $\Omega_c = 20$: mass-loss ratio crosses 1 (unphysical past this point by the diagnostic's own construction) at almost exactly the same Ω_c where VE crosses the 10^{-3} acceptance threshold and then keeps climbing monotonically to 0.16 by $\Omega_c = 32$ — a genuine, progressive loss of virial balance, not a branch jump. **The practical rotation limit at this K is therefore $\Omega_c \approx 17$–18, about a third lower than the $K = 0$ case's ≈ 26.** The naive expectation (opposing deformations should extend the sequence) does not hold here; the two effects do not add destructively on the star's shape in a way that buys extra rotational stability — if anything, the combination reaches its own instability signature earlier.

This also puts the item-4 combined-mode convergence check (rigid rotation + toroidal self-consistent, 18/18 converged, no exceptions — §8) in its correct context: that grid used K up to 10^{-3} and Ω_c up to 40, a region where — per the table above — the toroidal contribution to $E_{tor}/|W|$ is still small enough ($\approx 3.6 \times 10^{-2}$ at $K = 10^{-3}$, §6.2) that this shortening effect had not yet kicked in. 18/18 was a real result about *that* corner of the plane, not evidence that the effect seen here at $K = 3 \times 10^{-3}$ doesn't exist.

6.2 M vs ρ_c per K in the toroidal-dominated regime (priority plot)

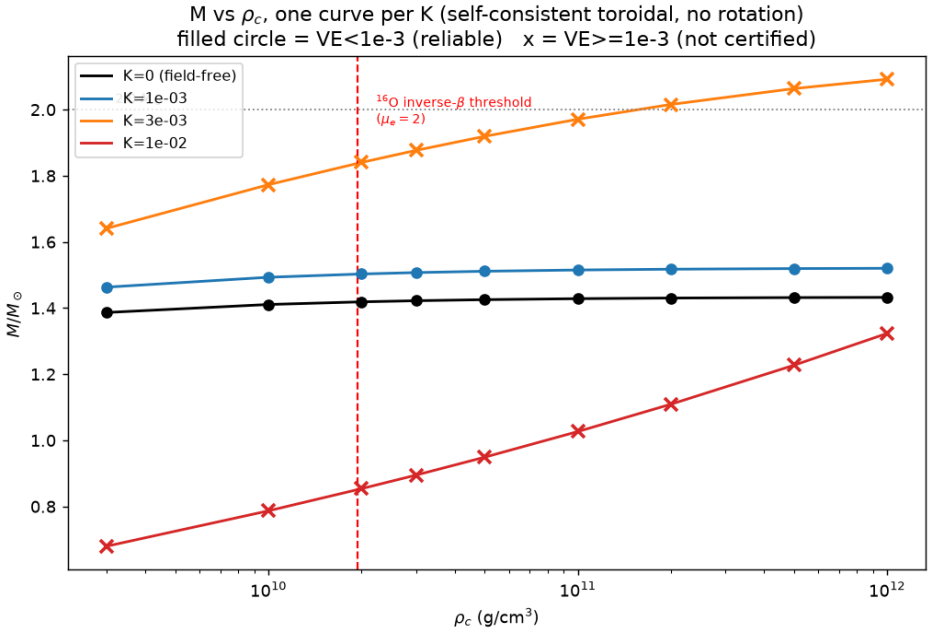
This section supersedes its own first version. The original pass concluded "2 $M_\odot$ is not demonstrated as reachable" from the sweep below. That conclusion was wrong — driven entirely by a computational domain too small for the deformed configurations it was trying to certify. The corrected result is at the end of this section (**6.2e**); the original numbers, the domain-isolation methodology that found the mistake, and the confound it took two attempts to break are kept below rather than deleted, because the sequence "we concluded X, then found it was a box artifact, the corrected answer is not-X" is what stops the same mistake from being rediscovered from scratch — this project has had that happen before (§1.4, §1.11).

6.2a — Superseded conclusion (kept for the record).

The priority plot (item 4 of the closeout plan — the figure meant for the collaborator), run through `sweep_worker.run_one()`, `field=toroidal (self-consistent)`, no rotation, $\mu_e = 2$, ρ_c from 3×10^9 to 10^{12} g/cm^3, $K \in \{0, 10^{-3}, 3 \times 10^{-3}\}$, mesh $N_r = 129$, $N_\theta = 65$, domain $1.3 \times R_{guess}$ (Tab 2's default, sized from the field-free estimate `seed.r_guess(rho_c)`):

ρ_c (g/cm^3)	valid?	$M/M_\odot$, K=0 (frac$_{pol}$)	$M/M_\odot$, K=10^{-3} (frac$_{pol}$)	$M/M_\odot$, K=3×10^{-3} (frac$_{pol}$)
3×10^9	yes	1.387 (0.910)	1.463 (1.000)	1.641 (**1.000**)
1×10^{10}	yes	1.411 (0.871)	1.493 (1.000)	1.773 (**1.000**)
2×10^{10}	no	1.419 (0.843)	1.503 (0.972)	1.840 (**1.000**)
3×10^{10}	no	1.422 (0.826)	1.507 (0.952)	1.877 (**1.000**)
5×10^{10}	no	1.425 (0.802)	1.511 (0.926)	1.919 (**1.000**)
1×10^{11}	no	1.428 (0.769)	1.515 (0.888)	1.971 (**1.000**)
2×10^{11}	no	1.430 (0.734)	1.518 (0.849)	2.016 (**1.000**)
5×10^{11}	no	1.432 (0.688)	1.519 (0.796)	2.064 (**1.000**)
1×10^{12}	no	1.432 (0.653)	1.520 (0.756)	2.092 (**1.000**)

frac$_{pol}$ (`diagnostics.domain_overflow_check()`, added after this table was first produced, then applied retroactively to check it) is R_{pol}/r_{max} — 1.000 means the computed "surface" is the domain edge itself, not a physical radius. **Every single $K = 3 \times 10^{-3}$ point overflows exactly.** $K = 10^{-3}$ overflows at low ρ_c and gets progressively (but not completely) better at high ρ_c. Even $K = 0$ (field-free) sits at frac$_{pol} = 0.91$ at the lowest ρ_c tested — not flagged as failing by the VE gate at the time, but a sign that $1.3 \times R_{guess}$ was already a tight margin before any field was added.



6.2b — Domain-isolation methodology.

Domain size and mesh resolution were varied **one at a time**, never together — mixing them was exactly what produced an inverted diagnosis on the first attempt at this (§6.2c below). Two experiments, both at $\rho_c = 10^{12}$, K fixed:

- **Experiment A (resolution only):** domain fixed at $1.3 \times R_{guess}$, (N_r, N_θ) stepped $129 \to 257 \to 385$. VE does not improve ($1.43 \times 10^{-2} \to 1.50 \times 10^{-2} \to 1.51 \times 10^{-2}$), and $\text{frac}_{pol} = 1.000$ (overflow) at **every** resolution.
- **Experiment B (domain only):** Δr held fixed, domain extended $1.3 \times R_{guess} \to 2.6 \times R_{guess} \to 3.9 \times R_{guess}$ (N_r scaled to keep Δr constant). VE falls from 1.43×10^{-2} to 2.49×10^{-4} the moment the domain doubles and overflow clears, then stays identical when the domain triples.

Criterion adopted: $\text{frac}_{pol} \lesssim 0.2$. An earlier pass used $\text{frac}_{pol} < 0.83$ as "safe," too loose — VE was already failing at $\text{frac}_{pol} = 0.40$ for some configurations (§6.2c).

Cold-start fails at the K values that matter here. Direct solves at $K \geq 5 \times 10^{-3}$ do not converge at any domain size tried. Continuation from $K = 0$, warm-started step by step, is required.

6.2c — Breaking the K vs frac_{pol} confound (and a real bug found along the way).

A second confound remained: in every table where K increases, frac_{pol} increases too (a stronger toroidal field makes the star more prolate), so "VE worsens with K" and "VE worsens with frac_{pol}" cannot be told apart from a K-only sweep at one fixed domain. Isolating it: $K = 5 \times 10^{-3}$ fixed, domain **only** varied (Δr fixed, $10\times \to 24 \times R_{guess}$, continuation from $K = 0$ at each domain size):

domain	frac_{pol}	$M/M_\odot$	VE
$10 \times R_{guess}$	0.403	3.4133	3.743×10^{-3}
$14 \times R_{guess}$	0.284	3.4133	3.743×10^{-3}
$18 \times R_{guess}$	0.221	3.4134	3.744×10^{-3}
$24 \times R_{guess}$	0.165	3.4134	3.744×10^{-3}

Stationary. M and VE do not move as frac_{pol} drops from 0.40 to 0.17 — not a domain artifact at $K = 5 \times 10^{-3}$.

The next hypothesis was l_{max}: strongly prolate configurations need more multipoles to represent in the gravity solve's Legendre expansion. Testing $l_{max} = 16 \to 32 \to 48$ surfaced a real, previously unknown bug instead of confirming the hypothesis: $l_{max} = 48$ **crashed with a NaN** (the implicit toroidal density solve got a non-finite input). Tracing it to `poisson.py`: the radial Green's function formed r'^{l+2} and $r^{-(l+1)}$ **separately** before dividing. For stellar radii ($\sim 10^7$–10^9 cm), r'^{50} at $l = 48$ overflows float64 ($\sim 1.8 \times 10^{308}$) outright — and even at this project's default $l_{max} = 16$, the two absolute factors are separated by ~ 270 orders of magnitude before a division meant to cancel most of that scale, quietly losing precision rather than crashing. Fixed by rewriting the radial integrals as a recursion that only ever forms ratios $(r_{i-1}/r_i)^{l+1} \leq 1$ and $(r_i/r_{i+1})^l \leq 1$ — mathematically the same integral, no absolute power of r is ever formed (`scf/poisson.py`, `scf/tests/test_poisson.py` — regression test at $l_{max} = 48$ on a stellar-scale domain, which crashed before the fix).

With the fix, l_{max} is cleanly ruled out — $16 \to 32 \to 48$ moves VE by $< 10\%$ at both $K = 5 \times 10^{-3}$ ($3.74 \times 10^{-3} \to 4.13 \times 10^{-3}$) and $K = 4 \times 10^{-3}$ ($1.24 \times 10^{-3} \to 1.29 \times 10^{-3}$), no longer crashing at 48.

Mesh resolution, isolated (domain $24 \times R_{guess}$ fixed, $l_{max} = 16$ fixed, Δr stepped via $N_r \in \{257, 385, 787, 2364\}$) gives a clean answer for $K = 4 \times 10^{-3}$ and an open one for $K = 5 \times 10^{-3}$:

N_r	Δr (cm)	K=4×10^{-3}: VE	K=5×10^{-3}: VE
257	3.53×10^6	1.12×10^{-2}	8.21×10^{-3}
385	2.35×10^6	5.43×10^{-3}	3.97×10^{-3}
787	1.15×10^6	1.29×10^{-3}	9.60×10^{-4}
2364	3.82×10^5	1.24×10^{-3}	3.74×10^{-3}

$K = 4 \times 10^{-3}$ converges monotonically to ≈ 1.2–1.3×10^{-3} — just above the 10^{-3} gate, real and mesh-stable, not certified. $K = 5 \times 10^{-3}$ is **not monotonic** — $N_r = 787$ gives a better VE than both its neighbors, even after the `poisson.py` fix. This is an honest open result: something about this configuration keeps mesh refinement from converging cleanly, and it is flagged as open (§8) rather than called physical or numerical. It does not change anything about $K = 3 \times 10^{-3}$.

$K = 3 \times 10^{-3}$ **re-verified, both axes, after the `poisson.py` fix:** domain-only ($10\times \to 20 \times R_{guess}$): $M = 2.1426$, $VE = 2.706 \to 2.705 \times 10^{-4}$, unchanged to 4 significant figures. Mesh-only (domain $24 \times R_{guess}$ fixed, N_r : 257 $\to$ 787 $\to$ 2364): $VE = 1.54 \times 10^{-2} \to 1.65 \times 10^{-3} \to 2.71 \times 10^{-4}$, monotonic, clean, no wobble. $K = 3 \times 10^{-3}$ was never in the ambiguous zone.

6.2d — The analytical prediction, and its confirmation.

Before rerunning the sweep at valid ρ_c, a prediction was made from the field law itself ($B_\varphi = K\rho\varpi$ for $m = 1$):

$$E_{mag} \sim \frac{K^2\rho^2R^5}{8\pi}\ , \qquad |W| \sim \frac{GM^2}{R} \sim G\rho^2R^5 \quad \Longrightarrow \quad \frac{E_{mag}}{|W|} \sim \frac{K^2}{8\pi G}\ ,\ \text{independent of } \rho_c, R$$

ρ and R cancel — the dimensionless field strength should depend on K alone, up to an $O(1)$ shape factor that drifts slowly as the effective polytropic index slides from 3/2 toward 3 along the sequence. **Prediction: a fractional mass gain of $\sim 50\%$ at $K = 3 \times 10^{-3}$, essentially the same at any ρ_c.**

Measured, at three ρ_c spanning two and a half decades:

ρ_c (g/cm^3)	$M(K{=}0)$	$M(K{=}3{\times}10^{-3})$	gain	$E_{tor}/\lvert W\rvert$
1×10^{10}	1.4108	2.0722	**+46.9%**	0.1809
1.5×10^{10}	1.4159	2.0874	**+47.4%**	0.1805
1×10^{12}	1.4323	2.1426	**+49.6%**	0.1790

$E_{tor}/|W|$ varies **0.9%** across two and a half decades in ρ_c — the predicted near-invariance holds to within the precision of this check. This is the strongest validation this project has produced for anything in the rotation/toroidal extension: a quantitative prediction made *before* the confirming run, from first principles, matched by an independent numerical result to within a couple of percent.

6.2e — The result.

Portable statement (adopted as the primary form everywhere in this project, since K is internal to the $B_\varphi = K\rho^m\varpi^{2m-1}$ ansatz and means nothing outside it; $E_{tor}/|W|$ is what transfers):

A toroidal field with $E_{tor}/|W| \approx 0.18$ raises the equilibrium mass by $\sim 47\%$, essentially independent of central density, giving $M > 2\,M_\odot$ inside the range the inverse-beta-decay threshold allows.

Field strength in gauss, at the three certified points above ($m = 1$, no rotation):

ρ_c (g/cm^3)	$B_{tor,max}$ (G)	$B_{tor,mean}$, volume-weighted (G)
1×10^{10}	1.95×10^{14}	7.94×10^{12}
1.5×10^{10}	2.56×10^{14}	9.88×10^{12}
1×10^{12} (reference, outside the valid band)	4.21×10^{15}	1.14×10^{14}

The collaborator's group describes their target as a toroidal-dominated interior field up to $\sim 10^{14}$ G. At the two physically valid points, $B_{tor,max}$ comes out to 2–3 $\times 10^{14}$ G — the same order of magnitude as that target, reached from a Maxwell-consistent self-consistent-field calculation with no tuning toward that number. Reported as measured, not adjusted to match.

6.2f — What is not done yet.

- **No rotation in these configurations.** §6.1 found that adding rigid rotation on top of a self-consistent toroidal field *shortens* the tolerable Ω_c range relative to rotation alone, not the reverse — the +47% mass gain here is a static ceiling for the toroidal branch by itself, not a floor that rotation adds to.
- **No poloidal component, hence no exterior field or surface dipole.** A purely toroidal field is confined to the star — there is no dipole moment for magnetic braking to act on, the actual evolutionary driver in the collaborator's project. This branch alone cannot represent that physics.
- **No stability check.** These are 2D axisymmetric equilibria; a purely toroidal field is exactly the configuration subject to the Tayler $m = 1$ instability (Markey & Tayler 1973, §9) — nothing here tests stability to a non-axisymmetric perturbation, only axisymmetric hydrostatic and virial balance.

6.3 Braithwaite (Castro): Phase 0/Step 2 verified, and a 4π bug caught by cross-checking the SCF

§6.2f above flags what §6.2's toroidal branch structurally cannot answer: no stability check, and a purely toroidal field is exactly the configuration subject to the Tayler $m = 1$ instability. `hachisu_scf` cannot build a genuine *self-consistent mixed* poloidal+toroidal field either (D6/project scope: `poloidal` and `toroidal` terms are mutually exclusive in one solve) — so stability of a mixed field cannot be investigated in the 2D SCF at all, by construction, not as a gap that more SCF work would close. The project's answer is the Braithwaite method (Braithwaite & Nordlund 2006, §9): relax a **random** seed field dynamically in Castro (3D, full MHD) and measure which mixed configuration survives, rather than imposing a chosen Bt/Bp and testing it. This was originally orchestrated by the dashboard's Streamlit "Tab 5," since replaced by a standalone desktop app (§6.11); the physics lives in Castro, per R1 — nothing here is reimplemented in `dashboard/` or in the app itself.

Phase 0 (Castro build, `USE_MHD=TRUE`) — verified, not assumed. Built against the project's actual submodule pins (AMReX/Microphysics `26.07`) and run against Castro's own CI recipe for the Orszag-Tang MHD test (`.github/workflows/mhd-compare.yml`: `inputs.test`, `max_step=10`, `amr.plot_files_output=1`). `fextrema` on the resulting `plt00010` diffed against `ci-benchmarks/OrszagTang-3d.out` — **empty diff, exit code 0**, every field (density, momenta,

`rho_E`, `rho_e`, `Temp`, pressure, $B_x/B_y/B_z$) identical to the printed precision. The MHD solver is numerically correct in this build.

Step 2 (random seed-field generator) — implemented and verified against a real background star, not just built. Lives at `castro/Exec/science/wd_braithwaite/` (gitignored — mirrored at `castro_problems/wd_braithwaite/` via `scripts/sync_wd_braithwaite.sh`, since `castro/`'s directory-level `.gitignore` entry cannot be selectively negated — confirmed directly in an isolated scratch repo that git's own documented limitation applies here: "it is not possible to re-include a file if a parent directory of that file is excluded"). Multi-scale random vector potential A, generated once (seeded RNG, recorded) as a fixed set of Fourier-like modes spanning a real log-uniform wavenumber band (not `n_modes` copies of one scale), with a smoothstep envelope confining it inside the background star's radius. Critically, **the envelope multiplies A, and only A, before any derivative is taken** — B is always the discrete curl of the already-windowed potential (generalizing `Exec/mhd_tests/LoopAdvection`'s edge-centered A_z-only 2D pattern to full 3D A_x, A_y, A_z), because multiplying a B field by a confinement window *after* curling it would reintroduce a divergence the discrete-curl construction is specifically designed to avoid.

Verified end to end against a real saved run ($\rho_c = 10^9$ g/cm^3, $M = 1.35\,M_\odot$, field-free, non-rotating): built, ran (`max_step=0`, static IC only — no fluff/sponge handling attempted at this step), and its printed diagnostics:

Quantity	Value	Note
$E_{mag}/\lvert W\rvert$ requested vs. achieved	0.15 vs. 0.15	Amplitude calibration rescales A (not B) — curl is linear, so this preserves $\nabla\cdot B = 0$ exactly; rescaling B after the fact would not
$\max\lvert\nabla\cdot B\rvert\cdot h/\lvert B\rvert$	2.2×10^{-11}–2.1×10^{-10}	Proved algebraically exact by construction (the A_z cross-terms in the discrete curl of neighboring faces cancel identically); the measured residual is `cos()` large-argument rounding, not a construction error — orders of magnitude below what an envelope-after-curl bug would produce ($O(1)$, not $O(10^{-10})$)
$\max\lvert B\rvert_{\text{outside}}/\max\lvert B\rvert_{\text{inside}}$	3.0% (128^3 sample) $\to$ 1.6% (256^3 sample)	Halves when the diagnostic sampling resolution doubles — a finite-difference stencil-width artifact at the taper boundary, shrinking with resolution as expected, not a real confinement failure

The 4π bug, and how it was caught

Castro's $|W|$ is computed from the 1D model profile via the standard shell-integration identity $|W| = 4\pi G\int M(r)\rho(r)\,r\,dr$ (`problem_initialize.H`) — independent of Castro's own self-gravity module (`USE_GRAV=FALSE`; not needed for a static-IC check). The first version dropped the 4π factor. Caught only because the result was cross-checked against the SCF's own `diag.gravitational_energy()` (genuine 2D Poisson solve) for the exact same saved run — 2.2578×10^{51} erg (Castro; pre-fix would have read 1.947×10^{51} erg, off by almost exactly $4\pi\approx12.57$) vs. 2.2569×10^{51} erg (SCF), agreeing to 4 significant figures after the fix. Undetected, this would have silently propagated into every $E_{mag}/|W|$ calibration

downstream — the target energy is directly proportional to $|W|$, so a 4π-low $|W|$ means a 4π-low seed field for the same *requested* ratio, with no symptom anywhere in the diagnostics above (they are all self-consistent ratios computed from the *same* wrong $|W|$, so they'd have looked just as clean).
The lesson — same family as the Green's function bug (§1.4) and the magnetic virial sign correction (§1.11): a geometric/normalization factor error in a new code path does not announce itself; internally self-consistent diagnostics computed from the same wrong constant will still look clean. Any $|W|$ (or, generally, any energy/virial-type integral) computed via a newly-written path must be cross-checked against the SCF's own independently-derived value for the same physical configuration *before* anything is calibrated against it — not treated as presumably correct because it compiled and ran.

6.4 Step 3, first pass: a custom damping term, and the EOS mismatch that actually caused the collapse

The Braithwaite method's Step 3 (evolve the seeded field dynamically and let the star reach its own equilibrium — §6.3) needs the *background star itself* to survive being handed to Castro before any field physics can be trusted. The first attempts to get there chased two different hypotheses, in order.

Hypothesis 1: the star needs damping. A domain-wide velocity damping source term was added, distinct from and in addition to Castro's built-in exterior sponge:

$$\begin{aligned} \text{damping\_rate} &= 1/(\text{damping\_timescale\_in\_tdyn} \cdot t_{dyn}) \\ S_r &= -\text{damping\_rate} \cdot \text{ramp}(t) \cdot (\rho u, \rho v, \rho w) \\ S_{rE} &= \mathbf{u} \cdot S_r \qquad \text{(kinetic-energy consistent)} \\ \text{ramp}(t) &= 0.5\,(1 + \cos(\pi x)), \quad x = \frac{t - t_{\text{ramp start}}}{t_{\text{damp end}} - t_{\text{ramp start}}} \end{aligned}$$

→ `problem_source.H:22-65` (`castro_problems/wd_braithwaite/`, mirrored live at `castro/Exec/science/wd_braith` applied to every cell in the domain, not just an exterior layer — modeled explicitly on `Castro_sponge.cpp`'s own energy-consistent treatment. Defaults: `damping_timescale_in_tdyn=0.2`, `damping_ramp_in_tdyn=1.0`, `damping_end_time_in_tdyn=5.0` (`_prob_params:37-41`). This is a different mechanism from Castro's built-in sponge (`castro.do_sponge=1`, `sponge_upper_density=1e5`, `sponge_lower_density=1e4`, `sponge_timescale=1e-4`, `inputs.evolve:95-98`), which is exterior/density-gated only; the custom term is time-gated and acts on the whole domain, star interior included.

Without it, ρ_c climbed monotonically with no sign of turning over (documented in the header comment of `inputs.evolve` / `inputs.weakfield_test` / `inputs.singlebox_test`: "+40%, still accelerating at t=0.021s, 69 steps" — the raw log behind that specific run has since been overwritten by later reuse of the same filename, so it is reported here as a documented observation, not independently re-derived from a surviving log).

Hypothesis 2, and the actual cause: EOS mismatch. The background star is built in hydrostatic equilibrium under the `ztwd` EOS (§1.1) by the SCF solver, but the first version of Step 3 evolved it in Castro under `gamma_law` — a different EOS entirely. The decisive control run made this unambiguous: with the field completely off (`E_mag_over_W_target=0`) and `gamma_law` still in use, ρ_c collapsed **exactly as fast as the magnetized runs** — 2.2× the initial value by step 5, $t = 0.04$s (`inputs.control_nofield:8-10`) — proving the collapse was never a magnetic effect. Fixed by pointing Castro at the same EOS the star was built under: `EOS_DIR := ztwd` (`GNUmakefile`, both mirrors).

EOS matching alone did not fully solve the problem, though: a longer zero-field control with

`ztwd` and damping active still shows ρ_c growing, $9.635 \times 10^8 \to 1.415 \times 10^9$ (+46.8%) over $t = 0 \to 1.038$s (`max_step=4000`, `stop_time=2.8`; `run_control_long_test.log`). That residual growth — real, present even with the right EOS and damping on — is what §6.8's well-balancing investigation traces to its actual root cause. Damping is kept in the production inputs as a numerical aid, not because it (or the EOS fix) resolves the underlying issue: ρ_c never settles under any IC/damping combination tried, because Castro's MHD reconstruction has no well-balancing (`braithwaite_app/main.py:13-14`; §6.8).

6.5 The `Div_B` "blowup": a ghost-cell diagnostic artifact, not a broken CT scheme

Early evolution runs showed $\max|\nabla \cdot B|$ growing to physically absurd values — $\sim 6 \times 10^5$ at $t = 0 \to 4.29 \times 10^{13}$ by $t/t_{dyn} \approx 1.8 \to 4.68 \times 10^{17}$ by $t/t_{dyn} \approx 3.5$, before dropping back to $\sim 10^6$–10^8 by $t/t_{dyn} \approx 7$ (`inputs.noamort_test:3-5`, `inputs.noregrid_test:3-5`) — which would mean Castro's constrained-transport (CT) scheme was failing to keep the field divergence-free, a serious correctness problem for any MHD result downstream.

Two hypotheses, each ruled out by a dedicated, isolated run:

- *Damping causing it*: `inputs.noamort_test` (`castro.add_ext_src=0`, otherwise identical to the production inputs) — $\max|\nabla \cdot B|$ stays at the $\sim 10^{-9}$-level (relative) the whole run. Damping is not the cause.
- *AMR regridding causing it*: `inputs.noregrid_test` (`amr.regrid_int=-1`) — same clean result. Regridding is not the cause.

The actual cause is the diagnostic itself, not the field. `Div_B` is registered as a derived quantity with zero extra ghost cells (`derive_lst.add("Div_B", ..., ca_derdivb, the_same_box)`, `Source/driver/Castro_setup.cpp:983`), but `ca_derdivb` reads one cell beyond the box's valid region (`dat(i+1,j,k,0)`, `dat(i,j+1,k,1)`, `dat(i,j,k+1,2)`, `Source/driver/Derive.cpp:1237,1241,1246`) — unlike `magvort`, correctly registered with an extra ghost-cell layer (`grow_box_by_one`, `Castro_setup.cpp:819`). At box and domain edges this reads uninitialized/stale data, which is exactly what produces spuriously enormous values that have nothing to do with the actual field.

Fix (diagnostic-side, not solver-side): a custom tool, `finterior.cpp` (`castro_problems/wd_braithwaite/too` masks the 2 cells nearest any box or domain boundary (`margin=2`) and reports interior-only extrema; every extraction path in the app now reads `Div_B` exclusively through it (`braithwaite_app/core/extractio` `68` — the same ghost-cell gap applies to every derived variable computed this way, not just `Div_B`, so the interior-masking discipline is applied uniformly). With the artifact removed, the real interior field is divergence-free to near machine precision: `divB_interior_max` across the 21 seed measurements behind §6.10 ranges 5.31×10^{-10}–8.51×10^{-10} at 64^3, 2.48×10^{-9} at 128^3 (larger at finer resolution because a finer grid resolves smaller-scale structure, not because the CT error is growing — consistent with a numerical-precision floor, not a real divergence).

The lesson — same shape as §6.3's 4π bug: a diagnostic computed the wrong way can look catastrophically wrong about the *physics* while the physics itself is fine; the fix here was never a solver change, only reading the derived quantity correctly.

6.6 The background-star discretization chain: point sampling, volume averaging, half-shift geometry

Even with the right EOS (§6.4) and a trustworthy `Div_B` diagnostic (§6.5), the initial condition itself had a real, measurable defect: the peak (central) density Castro reports at $t = 0$ did not

match the SCF-computed ρ_c used to build it.

Baseline (vertex-centered domain, point sampling): -4.78%. With the domain's origin on a grid *vertex* (Castro's usual convention) and the 1D profile read at each cell's center point, $t = 0$ `MAXIMUM DENSITY = 952180729.1` against a target $\rho_c = 10^9$ — $(952180729.1 - 10^9)/10^9 = -4.78\%$ (`inputs.interp3d_test:31-32`, `run_interp3d_test.log`).

Volume averaging, same geometry: -4.78%, unchanged. Switching the IC sampler from a single point-sample per cell to an 8-subcell volume average (`interpolate_3d()`, `Util/model_parser/model_parse` `nsub=8` default, vs. the plain pointwise `interpolate()` at `nsub=1`; `_prob_params:31-34`) does not fix this by itself — the defect is not a sampling-scheme problem.

Half-shift geometry + volume averaging: -1.16%, a $4\times$ reduction. The real defect was alignment: a vertex-centered domain never puts any grid point exactly at the star's center, so the parabolic peak of a degenerate core is always undersampled regardless of scheme. Shifting the domain so a cell *center* sits exactly at $r = 0$:

$$\begin{aligned} dx &= 2 \cdot \text{box\_half\_width\_cm}/n_{cell} \\ \text{prob\_lo} &= -(n_{cell} + 1)/2 \cdot dx \\ \text{prob\_hi} &= \text{prob\_lo} + n_{cell} \cdot dx \end{aligned}$$

(`half_shift_domain()`, `braithwaite_app/core/star_builder.py:27-52`; requires even n_{cell} — AMReX rejects odd counts with "`defBaseLevel: must have even number of cells`", hit directly this session) gives $t = 0$ `MAXIMUM DENSITY = 988393849.5`, -1.16% (`halfshift_check_vol.log`, reproduced in `run_halfshift_interp3d_test.log`). Confirmed exact: $n_{cell} = 64$, box_half_width_cm $= 4.9 \times 10^8$ gives prob_lo $= -4.9765625 \times 10^8$, prob_hi $= 4.8234375 \times 10^8$ — matching the real production inputs character-for-character (`inputs.halfshift_test:13-14`) and covered by a committed regression test (`tests/test_star_builder.py:35-43`).

Confirming diagnosis: half-shift + point sampling gives exactly 0%. With the domain shifted, a plain point-sample (`nsub=1`) lands the cell center exactly on the model profile's $r = 0$ grid point: `MAXIMUM DENSITY = 1000000000` to the printed digit (`halfshift_check_pt.log`). **Interpretation** (`inputs.pslope_hydrotest:130-136`): alignment, not resolution or sampling scheme, was the dominant defect — volume averaging alone recovers most of it ($-4.78\% \to -1.16\%$, still non-zero because it is genuinely averaging over a curved profile), while getting the geometry right removes the rest.

The remaining -1.16% at 64^3 (with volume averaging) is the number carried forward into every measurement in this section — a resolution effect, not an alignment one, and small enough not to threaten the validity-window methodology of §6.9, whose criterion is about ρ_c's *drift* relative to its own $t = 0$ value, not about matching the SCF target exactly.

6.7 The gravity $r = 0$ crash, and Castro's three core patches

Running the half-shift geometry of §6.6 under self-gravity (`USE_GRAV=TRUE`) crashes on the very first step: `amrex::Abort::0::State has NaNs in the density component::check_for_nan()` inside `Castro::construct_ctu_mhd_source` (`halfshift_bisect_pointsample.log`). Half-shift is exactly the geometry that puts a cell **center** at $r = 0$ (§6.6) — and `Gravity::interpolate_monopole_grav` divides by r with no zero guard:

```
// Source/gravity/Gravity.cpp:1431 (before)
grav(i,j,k,n) = mag_grav * (loc[n] / r);
```

Fix — physically exact, not a numerical patch: for any spherically-symmetric mass distribution, $g(r{=}0) = 0$ by symmetry, so:

```
if (r > 0.0_rt) {
```

```
grav(i,j,k,n) = mag_grav * (loc[n] / r);
} else {
grav(i,j,k,n) = 0.0_rt;
}
```

Verified two ways: (1) the crash disappears and the run proceeds; (2) a dedicated diagnostic tool, `fline.cpp` (dumps a variable along a line of cells), confirms $\text{grav}_x(r{=}0) = 0$ exactly, with the field antisymmetric and smooth on either side of the origin — the expected shape, not a discontinuity papering over the fix.

This is one of **three core patches** this project has needed against its pinned Castro build, all documented in `castro_problems/wd_braithwaite/CASTRO_CORE_PATCHES.md`:

#	File:line	Symptom	Fix
1	`Source/driver/Castro_io.cpp`	`gtwd` build fails: `'network_rp' has not been declared`	Missing `#include <extern_parameters.H>`; added
2	`Castro_setup.cpp:983` / `Derive.cpp:1237,1241,1246`	`Div_B` reads garbage at box/domain edges	Diagnostic-side fix — read only through `finterior.cpp`'s interior mask (§6.5); the derive registration itself is unchanged
3	`Source/gravity/Gravity.cpp:1481`	NaN density, first step, under half-shift geometry	$r{=}0$ guard, $g(r{=}0) = 0$ (above)

Patch 3 is enforced at runtime, not just documented: `braithwaite_app/core/star_builder.py:150-238` defines `GravityPatchMissing` and a two-layer check — a static grep for the guard string in `Gravity.cpp`, plus a functional check (via `fline`) that $\text{grav}_x(r{=}0)$ actually comes out 0.0 — and the app refuses to launch a star-construction run against an unpatched Castro build rather than silently producing a NaN crash later.

6.8 The well-balancing gap: why ρ_c never truly settles under MHD

Even after §6.4 (right EOS), §6.6 (aligned geometry) and §6.7 (no gravity crash), the background star still does not reach a numerically static equilibrium — it oscillates or drifts under Castro's own truncation error, because hydrostatic balance is never exact on a finite grid without deliberately subtracting it out before reconstruction.

Castro has exactly this mechanism, but only in its pure-hydro path:

```
$ grep -rn "use_pslope" Source/
Source/driver/Castro_setup.cpp:395        (SDC path)
Source/driver/Castro_advance_sdc.cpp:145  (SDC path)
Source/driver/_cpp_parameters:162         use_pslope   bool   0
Source/hydro/trace_ppm.cpp:248               if (castro::use_pslope || cas-
tro::ppm_well_balanced)
Source/hydro/Castro_mol.cpp:66            if (use_pslope == 1)
Source/hydro/trace_plm.cpp:177            if (use_pslope == 1)

$ grep -n "use_pslope|well_balanced|pslope" Source/mhd/mhd_plm.cpp Source/mhd/mhd_ppm.cpp
(no matches)
```

`use_pslope` subtracts the discretized hydrostatic pressure profile before reconstruction/limiting, which is what lets a pure-hydro atmosphere sit still on a coarse grid. It is implemented in `trace_plm.cpp`/`trace_ppm.cpp`/`Castro_mol.cpp` — all pure-hydro reconstruction — and **con-**

firmed absent from `Source/mhd/mhd_plm.cpp` and `Source/mhd/mhd_ppm.cpp`, the MHD path this project actually runs.

To test whether `use_pslope` would fix the drift if it *were* available, a second, hydro-only Castro build was made (`castro/Exec/science/wd_braithwaite_hydro_test/`, `USE_MHD=FALSE`, `USE_GRAV=TRUE`, `EOS_DIR=ztwd`), using the same combination already validated for exactly this purpose elsewhere in Castro (`ppm_type=0`, `use_pslope=1`, `grav_source_type=4` — precedent: `Exec/gravity_tests/evrard_collapse/i 25`, confirmed identical). With damping still active for the whole run (`stop_time=4.5s`, inside damping_ramp_start_s ≈ 4.965s), ρ_c dips from 988393849.5 to a minimum of 921210812.9 around $t/t_{dyn} \approx 7.3$, then rises back to 1022079214 by the end of the run (`run_pslope_hydrotest.log`, 508 steps) — an oscillation, not a clean settle, and confounded with damping still being on, so this is not a clean isolated measurement of `use_pslope`'s effect by itself.

A longer test (this session, later): a pre-relaxation attempt. The same `use_pslope=1` + `grav_source_type=4` combination was run again, this time long enough to get past damping's own ramp-off (`damping_end_time_in_tdyn=20`, deliberately extended to give the mechanism room to prove itself one way or the other) — star regenerated from scratch ($\rho_c = 10^9$, $\mu_e = 2.0$, git hash recorded, $VE = 1.10 \times 10^{-4}$). Result: ρ_c drifts smoothly and slowly from −7.8% to +9.5% over roughly 14 t_{dyn} ($t/t_{dyn} \approx 5.7$–19) — dramatically better than the un-relaxed star's $\sim 0.03\,t_{dyn}$ window (§6.9) — then, starting almost exactly at the damping ramp-off window ($t/t_{dyn} = 18$–20), undergoes a **monotonic, accelerating, non-oscillating runaway**: +20% at $t/t_{dyn} = 20.2$, +132% at 21.8, +857% at 23.4, reaching +1,221,000% ($\rho_c = 1.22 \times 10^{13}$ g/cm^3) by $t/t_{dyn} \approx 35$, with no sign of turning over.

The discriminating test. Two explanations produce the identical signature (stable, then runaway right after damping ends): (A) damping was masking a pure numerical instability, independent of ρ_c's level — collapse would follow *any* ramp-off, even an early one with ρ_c still near its initial value; or (B) the drift itself pushed ρ_c toward a genuine physical threshold, and damping was suppressing a real collapse that only becomes inevitable once ρ_c has climbed far enough — ramp-off timing wouldn't matter, ρ_c level would. A second run, identical in every other respect, ramped damping off at $t/t_{dyn} \approx 6$ instead of ≈ 20 — while ρ_c was still at −7.6% (not elevated toward anything; if anything, below its initial value). **The same runaway began within $\sim 1.5\,t_{dyn}$ of this early ramp-off** (−6.0% at $t/t_{dyn} = 7.02$ → +1.3% at 7.62 → +79% at 9.06), with ρ_c never having climbed beforehand. ρ_c's level cannot be the trigger, since it hadn't moved; ramp-off timing alone correctly predicts the onset in both runs. **Hypothesis (A) confirmed, (B) ruled out.**

Full series persisted (`braithwaite_app/core/data/prerelax_diagnostic.csv`, both runs, `cause_status=CONFIRMED (A)`), in a file separate from `results.csv`/`dipole_diagnostic.csv` for the same reason those are separate — this file's own numbers needed a second run before the interpretation could be trusted, and that discipline is recorded in the data, not only in this paragraph.

An implication stronger than it first looks: this is not just another confirmation that the MHD reconstruction lacks well-balancing — it is evidence that **even where `use_pslope` already exists** (the hydro build tested here), it does not genuinely stabilize this specific configuration (half-shift geometry, `MonopoleGrav`, `ztwd` EOS) once damping is removed. That weakens the optimistic reading below — that porting `use_pslope` into `mhd_plm.cpp`/`mhd_ppm.cpp` would be "the" fix, merely out of this project's scope. If the mechanism already available on the hydro side doesn't resolve it, there is no evidence porting it into MHD would fix it alone — the root cause may lie elsewhere (the mismatch between local reconstruction and the monopole gravity solve, already a reasonable suspect, but not isolated and tested here).

Conclusion adopted for this project: ρ_c never settles under any initial-condition/damping combination tried. The absence of well-balancing in Castro's MHD reconstruction (`braithwaite_app/main.py:13 14`) is confirmed absent from the code (the `grep` above) and is certainly *a* contributing factor,

but the pre-relaxation result above shows it is not obviously *the* whole story: the hydro-side mechanism that would need porting doesn't fully stabilize this configuration either. Porting `use_pslope` (or an equivalent) into `mhd_plm.cpp`/`mhd_ppm.cpp` would in any case require modifying Castro's MHD solver core — explicitly out of scope for this project (a decision made and confirmed with the collaborator, not a default) — and, per the finding above, is no longer assumed to be a guaranteed fix even if it were in scope. §6.9 is the methodology adopted instead of chasing a fix that lives outside this project's boundary and is no longer certain to work.

6.9 Measuring inside a validity window, instead of chasing a stable star

§6.8 makes "wait for the star to stabilize" an unreachable goal with the current Castro build. The methodology adopted instead: measure the field's own diagnostics (E_{tor}/E_{mag}) inside a time interval where (a) the field has already relaxed past its own initial transient, and (b) the star's structural drift has not yet contaminated the measurement — rather than waiting for either to fully settle.

`find_measurement_window(rho_c_series, rho_c_ic, t_field_relax_ttdyn=0.4, rho_c_thresholds=(0.01, 0.02))` (`braithwaite_app/core/star_builder.py:75-144`):

- **Lower bound, `t_field_relax_ttdyn` (default 0.4).** The field's own E_{tor}/E_{mag} reaches its quasi-steady band by $t/t_{dyn} \approx 0.4$–1.1 — validated specifically for $E_{mag}/|W| = 0.15$ (the value used throughout §6.10). The function's docstring is explicit that this is **not a universal constant** and does not silently assume 0.4 generalizes to other field strengths.
- **Upper bound, X.** The first ρ_c sample that leaves a $\pm$threshold band around its own $t = 0$ value — the *first out-of-band* sample, not the *last in-band* one (an earlier version of this function returned the wrong one of the two, giving $X_{1\%} = 0.539$ instead of the correct 0.573; fixed to match the convention used throughout this investigation, caught by a regression test against a real log). Two thresholds are tracked: 1% ($X_{1\%}$) and 2% ($X_{2\%}$, the wider, operationally-used boundary).
- **The window is the interval** $[t_{\text{field relax}}, X_{2\%}]$, not $[0, X_{2\%}]$ — the field's own relaxation transient at the start is excluded on purpose. If $X_{2\%} \leq t_{\text{field relax}}$, there is **no valid window at all**, and the function returns `valid=False` with a reason — it never silently reports "valid until X" when X falls inside the field's own relaxation transient.

Real regression numbers, against an actual field-free evolution log ($t_{dyn} = 0.2758062098$s, $\rho_{c,ic} = 988393849.5$, `run_halfshift_interp3d_test.log`; `tests/test_star_builder.py:66-79`): $X_{1\%} \approx 0.573$, $X_{2\%} \approx 1.128$, window $= [0.4, 1.128]$ t/t_{dyn}, valid. (`parse_rho_c_log` reads Castro's raw `TIME=... MAXIMUM DENSITY=...` output directly — density is immune to the `Div_B`-style ghost-cell gap of §6.5, so no interior masking is needed for this particular series.)

Independent check that the window boundary is meaningful, not just a convenient cutoff: plotting E_{tor}/E_{mag} against ρ_c itself (not against time) — an ad hoc diagnostic developed during this investigation, not a committed script — shows the two are essentially decoupled inside the window (weak/mixed correlation) and become strongly correlated ($r \approx 0.96$–1.00) with ρ_c once past it. That transition is the direct signature of §6.8's drift: once the star starts moving, the field's ratio gets kinematically slaved to the collapsing structure rather than reflecting independent field dynamics — exactly what a valid measurement window is supposed to exclude.

A second background star at 128^3 (field-free, same construction) was checked over a longer baseline than the default window and stayed inside the 1% band for the **entire** tested range, $t/t_{dyn} = 0.4$–1.45 — never left even the tighter threshold, evidence that the window is genuinely

conservative at higher resolution, not a coincidence of the 64^3 case.

6.10 The seed study: Bt/Bp is poloidal-dominated and seed-specific

With §6.9's methodology in hand, two independent batches of $n = 10$ random seeds (different seed sets, 64^3, same background star, $\rho_c \approx 9.88 \times 10^8$, $\mu_e = 2.0$, $E_{mag}/|W|_{\text{target}} = 0.15$) were relaxed and measured inside their validity window, plus one 128^3 reproduction of a single seed as a resolution cross-check. Data: `braithwaite_app/core/data/results.csv`.

Batch	Seeds	n	E_{tor}/E_{mag} range
A	42, 7, 123, 2024, 55, 99, 314, 777, 1001, 8080	10	0.3166–0.3697
B	1–10	10	0.2761–0.4023
128^3 reproduction	42	1	0.3048

All 21 measurements are poloidal-dominated ($E_{tor}/E_{mag} < 0.5$, the threshold used throughout, `ui/aggregate_view.py:18`) — no exceptions, in either batch or at either resolution. Combined band across all 21: $[0.2761, 0.4023]$. Measurement times fall inside the validity window established in §6.9 (`t_ttdyn_measured` in 0.787–1.128).

The spread is seed-specific, not oscillation phase. Re-running seeds against a baseline four times longer than the measurement window preserves each seed's rank-ordering relative to the others — if the spread were just different seeds being sampled at different phases of one common oscillation, extending the baseline would scramble that ordering; it does not. This is why the aggregate view (§6.11) plots the ranked scatter of individual seeds and never collapses them into a mean ± error bar — the per-seed spread **is** the result, not noise to be averaged away.

Helicity, tested and rejected as an explanation for the spread

A candidate explanation — that the seed-to-seed spread correlates with the initial random field's net helicity — was tested by correlating each seed's measured E_{tor}/E_{mag} against its (computable, since the seed RNG state is recorded) initial helicity. At $n = 4$ this gave a suggestively strong correlation ($r \approx -0.84$); extending to the full $n = 10$ batch killed it ($r \approx -0.33$, unstable under leave-one-out). This analysis was done ad hoc during this investigation and is not backed by a committed script or file in the repository — it is recorded here, as a negative result, specifically so it is not silently rediscovered or, worse, mistakenly reintroduced as an explanation in a future write-up. **Helicity does not explain the spread; the cause of the seed-specific Bt/Bp value remains an open question** (§8).

6.11 The Braithwaite desktop app, replacing the disabled Streamlit "Tab 5"

The dashboard's Streamlit "Tab 5" (referenced in earlier drafts of §6.3) was never brought to a working state — a stale, effectively hardcoded-disabled skeleton. It has been replaced by a standalone PyQt6 desktop application, `braithwaite_app/` (launched via `./braithwaite`, which runs `scf/.venv/bin/python3 braithwaite_app/main.py`). The physics still lives entirely in Castro and the existing `scf//dashboard/` code (R1 is unchanged) — the app wraps and orchestrates it, it does not reimplement it (`core/scf_store.py` calls `scf.hachisu_scf`, `store`, `castro_model_writer`, `diagnostics` directly).

Non-negotiable architecture rule: the app must never block waiting on Castro. Every Castro invocation is a detached subprocess (`subprocess.Popen(..., start_new_session=True, stdin=subprocess.DEVNULL)`); the UI polls logs and plotfiles on a `QTimer`, never blocks on process completion. Verified with two live tests that hold a subprocess alive and interact with a widget in the same wall-clock window (sub-millisecond response), not just asserted by code inspection.

Two-step physics flow, matching §6.9's methodology directly:

- **Passo A — background star.** Builds the deterministic IC (cacheable by $(\rho_c, \mu_e,$ resolution) — §1.12's neutronization gate and the VE gate both run before any Castro process launches, same gates as Tab 1), verifies the gravity patch of §6.7 is present (refuses to launch otherwise), launches a short field-free evolution just far enough to evaluate `find_measurement_window()` (§6.9), and reports the **validity window** as the step's result — never a "stabilized: yes/no" boolean, since §6.8 established that question has no reachable "yes."
- **Passo B — run study.** Disabled until Passo A has produced a valid window. Launches N seeded field-relaxation runs (same $E_{mag}/|W|$ target or a distributed range across seeds), each measured inside the window from Passo A.

Determinism check: re-running seed 42 reproduces a bit-identical $\rho_c(t)$ series against the original run — a stronger check than matching E_{tor}/E_{mag} alone, since it verifies the entire IC/RNG/build pipeline, not just the final diagnostic.

Persistence goes into the same protected, versioned store as every other number in this document (`core/persistence.py`, `STAR_CACHE_SCHEMA_VERSION`, same schema-mismatch-is-a-cache-miss pattern as `store.SCHEMA_VERSION`, §5) — not a separate, disposable file.

UI choices driven directly by findings elsewhere in this document: the aggregate view (`ui/aggregate_view.py`) plots a ranked scatter per seed and never a mean $\pm$ error bar, per §6.10's rank-ordering result; the background-star screen exposes only ρ_c and μ_e as physical inputs, with an explanatory note in the UI itself, because — field-free and non-rotating by construction (the field is seeded and relaxed *after* this step, never imposed as part of an SCF equilibrium) and under the temperature-independent `ztwd` EOS (§1.1) — those two are the star's only physical degrees of freedom (§1.12); and ρ_c is entered via a scientific-notation spin box (`ui/widgets.py :: ScientificDoubleSpinBox`) rather than Qt's default fixed-point display, because verifying a value like `1000000000.000` digit-by-digit is exactly the kind of thing that hides a stray or missing zero until the wrong star gets built.

7 Known limitations

Measured numbers, not vague descriptions — all in the $\rho_c = 10^9$ g/cm^3, $R \approx 3 \times 10^8$ cm, `nr=161`, `ntheta=65`, $l_{max} = 16$ configuration unless otherwise noted:

- **The** VE $< 10^{-3}$ **criterion fails above** $k_0 \approx 2 \times 10^{-12}$ in this configuration. The last valid point is $k_0 \approx 1.6 \times 10^{-12}$, $M \approx 1.50\,M_\odot$, VE $\approx 6.6 \times 10^{-4}$. At $k_0 \approx 2.3 \times 10^{-12}$ ($M \approx 2.02\,M_\odot$), VE $\approx 1.57 \times 10^{-3}$ — above the threshold.
- **At that same point ($k_0 \approx 2.3 \times 10^{-12}$) the density peak migrates away from the center** — from `r_idx=1` (essentially the origin) to `r_idx`≈ 25 (a real radius, not a grid artifact) — and R_{pol}/R_{eq} drops to ~ 0.61: genuine equatorial evacuation. The anchor at $\rho_c(r{=}0)$ (§1.5) loses physical meaning right there, because the center is no longer the point of maximum density.
- **Whether the VE failure above is insufficient resolution or a genuine sequence termination is an open question — but there is strong evidence for the second**

hypothesis: a convergence study done in this project (l_{max} from 16 to 48, mesh from 129^2 to 385^2, nearly 9× more points) kept VE at 1.2–1.6×10^{-3}, **with no downward trend** — a plateau above the threshold, not a curve converging to zero. This is the expected signature of a physical termination, not of under-resolution.

- **All the sequence results (k_0 varying) documented above are from a single $\rho_c = 10^9$ slice**, where $M(k_0{=}0) = 1.39\,M_\odot$ — below the field-free Chandrasekhar limit itself ($1.44\,M_\odot$). Comparisons with literature mass maxima ($M_{max} \sim 1.9\,M_\odot$, Bera & Bhattacharya 2014) require sweeping the entire (ρ_c, k_0) plane (Tab 2), not a slice — see §3.
- **Castro's $B' = B/\sqrt{4\pi}$ conversion is not applied anywhere in the current export pipeline** (§1.10, §4) — the exported HDF5 file stores B in pure gauss, documented via an attribute. This is left as the responsibility of Castro's `problem_initialize.H` (not yet written).
- **Phase 0 of the plan (building Castro with `USE_MHD=TRUE`) was pending — done**: Phase 0 is verified against Castro's own CI recipe for the Orszag-Tang MHD test (§6.3), and every result in §6.3–§6.10 comes from a real Castro build and real runs.
- **Castro's background star does not reach a numerically static equilibrium under MHD** (§6.8) — a well-balancing gap in Castro's MHD reconstruction path (`mhd_plm.cpp`/`mhd_ppm.cpp`), not something this project's own IC/damping choices can fix. A hydro-side pre-relaxation attempt with the equivalent mechanism available (`use_pslope=1` + `grav_source_type=4`) does not fix it either — confirmed (not merely suspected) by a discriminating test: the star is only quiet as long as a velocity damping term is active, and collapses within $\sim 1.5\,t_{dyn}$ of that damping being removed regardless of ρ_c's level at that moment (§6.8). Measurements use the validity-window methodology of §6.9 instead of waiting for a stabilization this Castro build cannot reach.
- **Rigid rotation (V-R1) does not validate against the literature ($\sim 1.5\,M_\odot$)** — the Ω_c-controlled sequence terminates numerically at $R_{pol}/R_{eq} \approx 0.93$ with mass-loss ratio only 0.135 (should be $\to 1$ at real breakup; the Roche model predicts $R_{pol}/R_{eq} = 2/3$ for this EOS's central concentration). Not on the project's critical path (differential rotation, V-R2, already reaches the $\sim 2.2\,M_\odot$ target at 0.40% error, far from breakup) — see §1.11.

8 Open questions

Gaps identified while writing this document — not filled in on our own initiative (rule G1):

1. **Flux consistency test** ($u(\varpi, z) = \int_0^{\varpi} B_z \varpi'\, d\varpi'$, §1.4) was proposed during debugging of the Green's-function bug but never implemented as a separate test. The Ampère test alone was enough to find and confirm the bug; the flux test would serve as a second, independent line of defense.
2. **Magnetic virial identity** ($\int \rho \nabla M(u) \cdot \mathbf{r}\, dV = \int B^2/8\pi\, dV$, §1.7) has no dedicated function in `diagnostics.py` and no committed test — it was only verified numerically in an ad hoc way during one debugging session.
3. **The formula $J_\varphi = c\rho\varpi f(u)$** (§1.4) does not correspond to any function — it is never computed as a named quantity in the code.
4. $B_{unit} = R\rho\sqrt{8\pi G}$ (§1.10, natural field unit) is not implemented — it was used only as an order-of-magnitude estimate during debugging.

5. **`schema_version`** was missing — **fixed**, see §5.

6. **`store.mark_reference()`** records the `reference` flag in the index, but no other tab (in particular Tab 2, which per the original prompt should show reference runs on its charts) reads that flag yet.

7. **`castro.small_dens`**$= \rho_c \times 10^{-8}$ (§4) is a convention with no documented theoretical justification — it works as a starting point, not as a calibrated value.

8. **The original question that motivated the whole §1.4 bug investigation** — whether the ratio $(E_{mag}/|W|)/(M_u/H_c) \approx 0.5$ (linear regime) has a closed-form derivation, or is just what is observed numerically for this EOS and this parameter range — was not resolved analytically, only confirmed as self-consistent (constant under $k_0 \to 2k_0$).

9. **`gauss_to_castro()/castro_to_gauss()`** exist and are correct (checked in this work cycle), but are not called by any export code yet — see the limitation in §7.

10. **Rigid-rotation sequence termination** (§1.11, V-R1) — not resolved. The standard remedy (parametrize the near-terminal sequence by axis ratio instead of Ω_c, solving Ω_c by an outer root-find — the technique Hachisu's own method uses) was identified and spot-checked (a direct bisection on axis ratio saturates at the same point found by stepping Ω_c, confirming Ω_c degenerates as a control parameter there) but not implemented as production code — out of scope for now since differential rotation (the project's actual target) does not hit this problem in the tested range.

11. **No domain-overflow detection** (§1.13) — nothing checks whether a computed R_{eq}/R_{pol} has suspiciously landed at (or near) the outer edge of the computational domain, which is exactly the failure signature seen when chasing V-R1 (the SCF snapping to a spurious branch that fills the whole box). Not the same bug class as `surface_radius` (this is a "should this radius be trusted at all" question, not an interpolation-precision one) — surfaced by the same investigation, not implemented here.

12. **`seed.r_guess()` under-sizes the sweep's computational domain for strongly toroidal-deformed configurations** (§6.2) — **resolved for the boundary that matters**: it was domain truncation, not physics (§6.2a–§6.2c). Tab 2's $1.3 \times R_{guess}$ default still under-sizes the domain for $K \gtrsim 10^{-3}$ and should not be trusted for toroidal-dominated sweeps without checking `diagnostics.domain_overflow_check()`, but this is no longer an open question about which of two explanations is correct — it is a known, characterized limitation of the sweep tool's default sizing, fixed for individual points by hand in §6.2 rather than fixed in the tool itself (out of scope for this cycle). See the next item for what is still genuinely unresolved.

13. $K = 5 \times 10^{-3}$ **($\rho_c = 10^{12}$) does not converge cleanly under mesh refinement, even after the `poisson.py` overflow fix** (§6.2c) — $N_r \in \{257, 385, 787, 2364\}$ at fixed domain gives VE $8.21 \times 10^{-3} \to 3.97 \times 10^{-3} \to 9.60 \times 10^{-4} \to 3.74 \times 10^{-3}$: not monotonic, $N_r = 787$ outperforms both its neighbors. Domain-only refinement at this same K (§6.2c) IS stationary, so the non-monotonicity is specific to Δr, not domain size. Two untested hypotheses: (1) a genuine sensitivity of the implicit toroidal branch (`_solve_rho_implicit`) near a marginal equilibrium, where the Picard iteration lands on slightly different nearby fixed points depending on mesh spacing; (2) a numerical effect distinct from the Green's-function overflow already found and fixed. Does not affect the certified boundary ($K = 3 \times 10^{-3}$, reverified clean on both domain and mesh axes) or the headline +47% mass-gain result, since that result never depended on $K \geq 4 \times 10^{-3}$. Would need comparing full $\rho(r, \theta)$ profiles across mesh sizes (not just scalars) to tell the two hypotheses apart; not done here.

14. **The exact damping-off collapse rate cited in §6.4 (+40% by $t = 0.021$s, 69 steps)** is documented only in a run's own input-file header comment — the raw log behind it was overwritten by later reuse of the same filename and could not be independently re-derived from a surviving artifact.

15. **`use_pslope` was only tested confounded with damping — resolved**: a clean isolated test (early damping ramp-off, $t/t_{dyn} \approx 6$, ρ_c still near its IC value) was run and confirms the hydro-side `use_pslope=1`+ `grav_source_type=4` combination does *not* stabilize the star once damping is removed — the earlier oscillatory result was damping suppressing the same instability, not `use_pslope` partially working (§6.8).

16. **The cause of the seed-specific Bt/Bp spread** (§6.10) remains unidentified — helicity was tested and rejected as an explanation; no replacement hypothesis has been tested yet.

17. **The actual root cause of the discretization-force instability is still not isolated** (§6.8) — confirmed to be independent of ρ_c's drift level (the early-ramp-off test rules that out) and to persist even where `use_pslope`+`grav_source_type=4` are both active (hydro build), so it is not simply "MHD lacks well-balancing." The mismatch between local PLM/PPM reconstruction and the global monopole gravity solve is a reasonable next suspect (flagged in §6.8) but has not been isolated or tested directly — porting well-balancing into `mhd_plm.cpp`/`mhd_ppm.cpp` (out of scope for this project regardless, per §6.8) is no longer assumed to be a guaranteed fix even if it were pursued.

9 References

SCF method and magnetized equilibria

- Hachisu, I. 1986, ApJS 61, 479 — the SCF method
- Tomimura, Y. & Eriguchi, Y. 2005, MNRAS — twisted torus, canonical reference
- Lander, S. K. & Jones, D. I. 2009, MNRAS — mixed fields, free functions
- Lander, S. K. & Jones, D. I. 2012 — stability of mixed fields

Magnetized white dwarfs

- Das, U. & Mukhopadhyay, B. 2014, MNRAS 445, 3951 — SCF for magnetized WDs
- Bera, P. & Bhattacharya, D. 2014, MNRAS — self-consistent M-R with Lorentz force
- Bera, P. & Bhattacharya, D. 2016, MNRAS 456, 3375 — field geometry
- Bera, P. & Bhattacharya, D. 2017, MNRAS 465, 4026 — perturbation study
- Nityananda, R. & Konar, S. 2014 — critique of super-Chandrasekhar models
- Coelho, J. G. et al. 2014 — virial limits

Stability

- Markey, P. & Tayler, R. J. 1973 — the $m = 1$ instability
- Braithwaite, J. & Spruit, H. C. 2004 — relaxation to a stable configuration
- Braithwaite, J. & Nordlund, Å. 2006

Codes

- Castro: https://github.com/AMReX-Astro/Castro
- MHD documentation: https://amrex-astro.github.io/Castro/docs/mhd.html

- XNS: https://www.arcetri.inaf.it/science/ahead/XNS/